k Nearest Neighbors

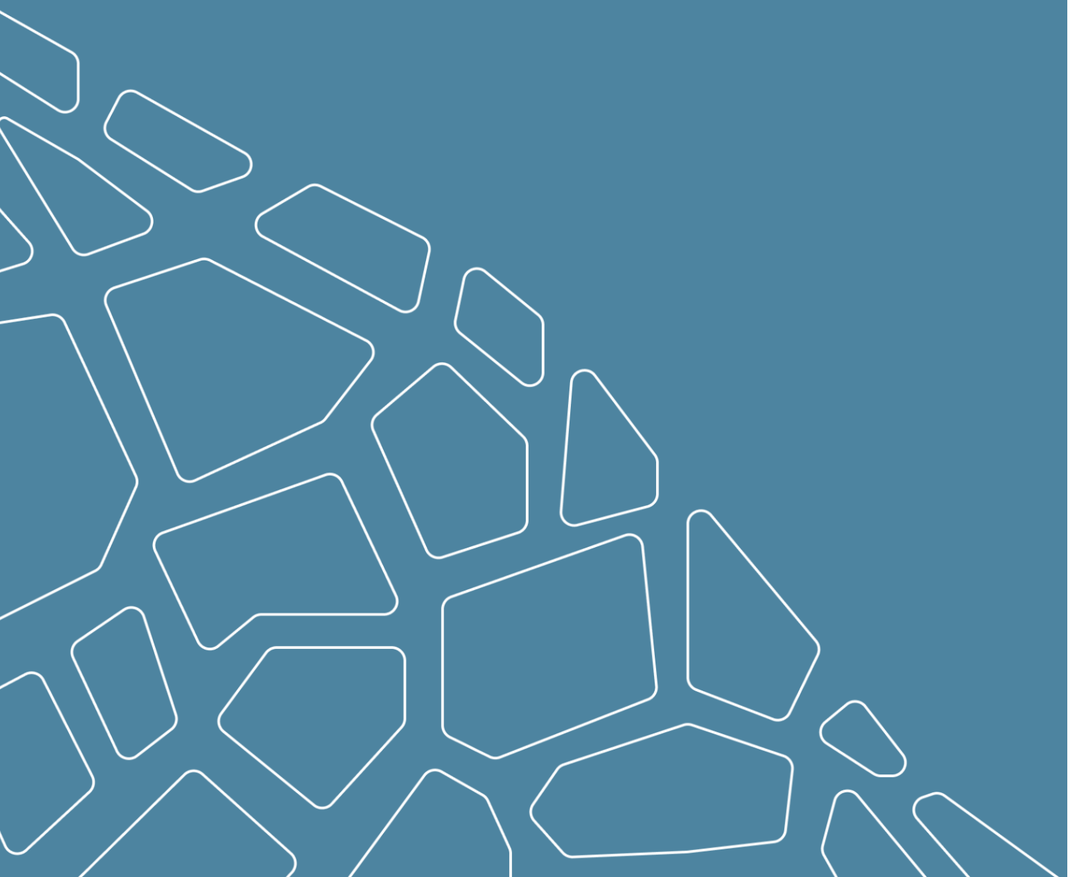
Vladislav Goncharenko
Senior researcher

girafe ai

ITMO, fall 2026

Recap

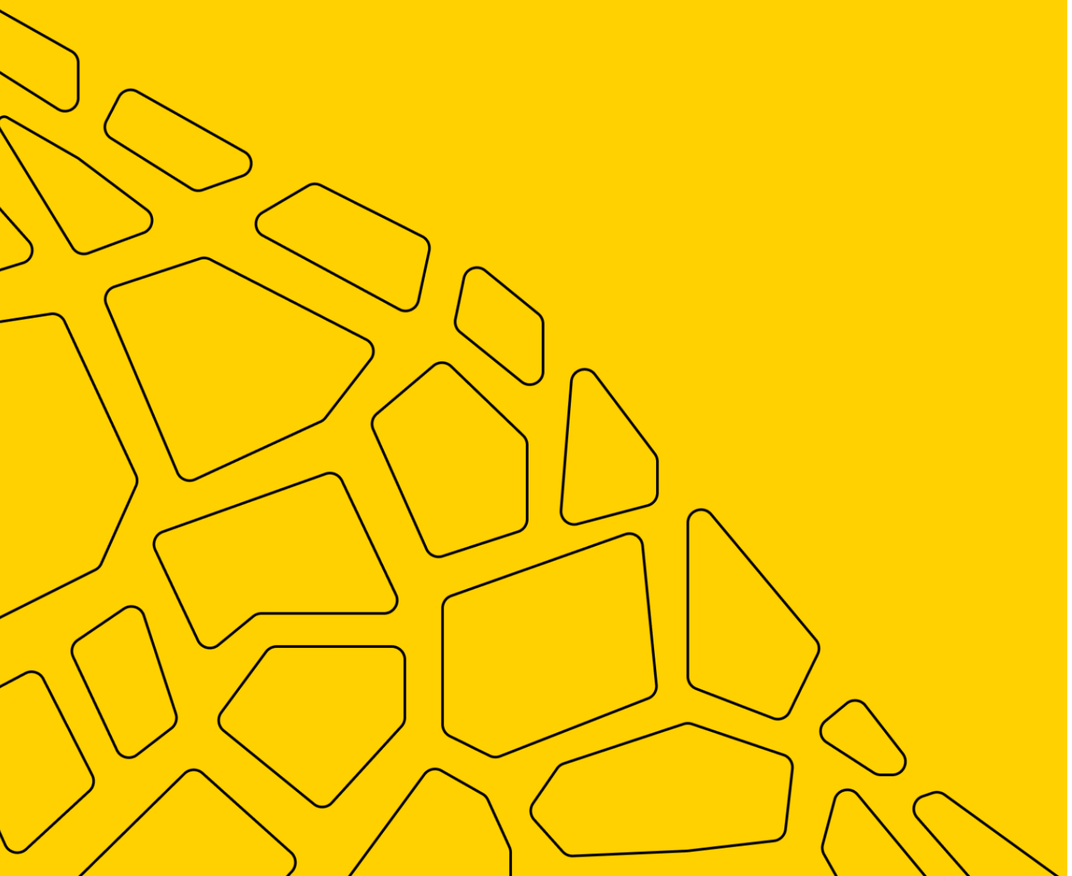
Lecture 4:
SVM, PCA



1. Support Vector Machine (SVM)
    a. Hinge loss
    b. Kernel trick
2. Dimensionality reduction and PCA
    a. Problem statement
    b. Singular Value Decomposition
    c. Eckart–Young theorem
    d. Variance based definition
3. Data normalization
    a. MinMaxScaler
    b. StandardScaler

Outline

Lecture 5:
kNN indexes



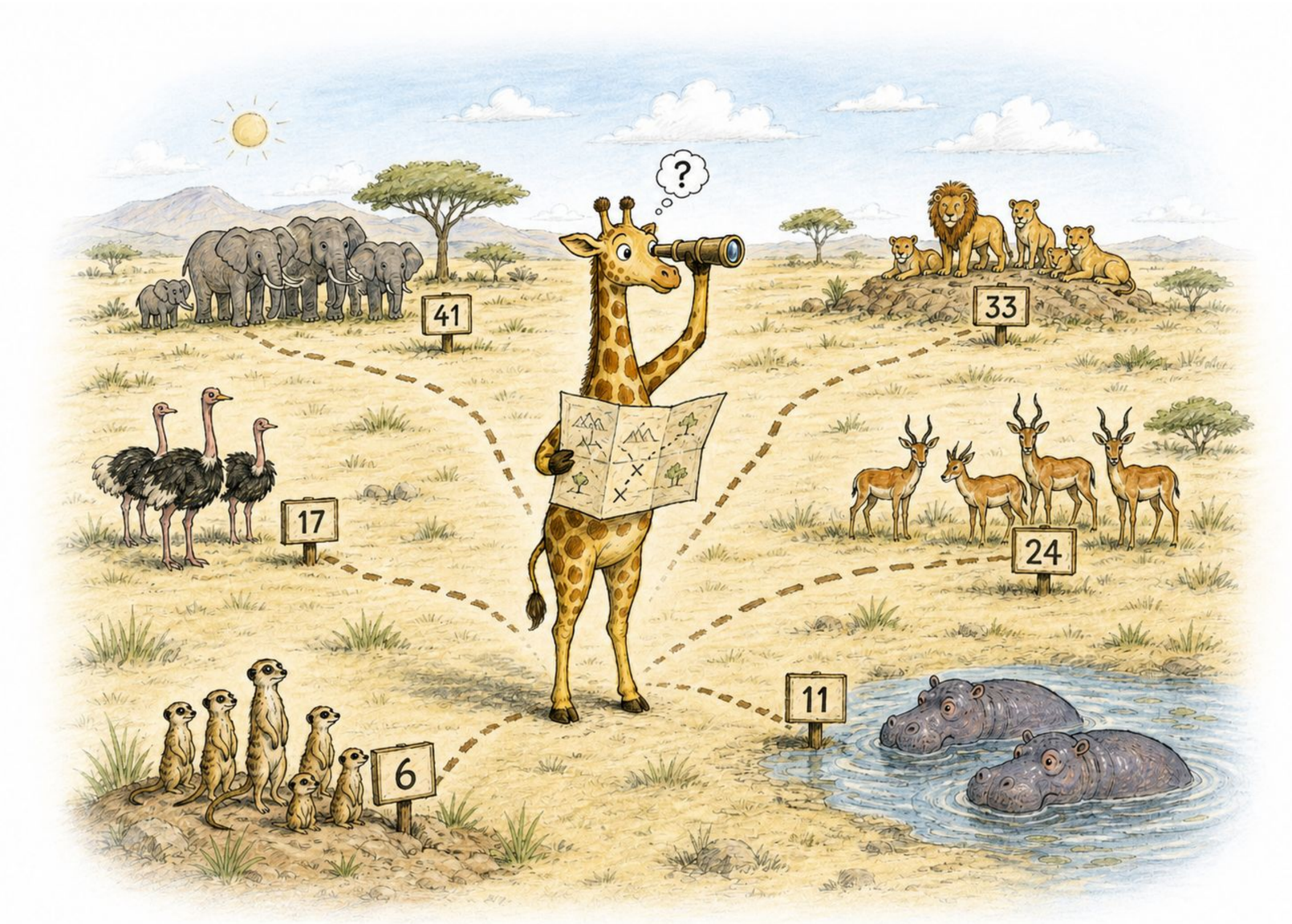
1. k Nearest Neighbors model
    a. contemporary use cases
2. kNN indexes
    a. ideas
    b. Annoy
    c. HNSW
3. Implementations
    a. FAISS
4. Vector databases
    a. Qdrant

k Nearest Neighbors

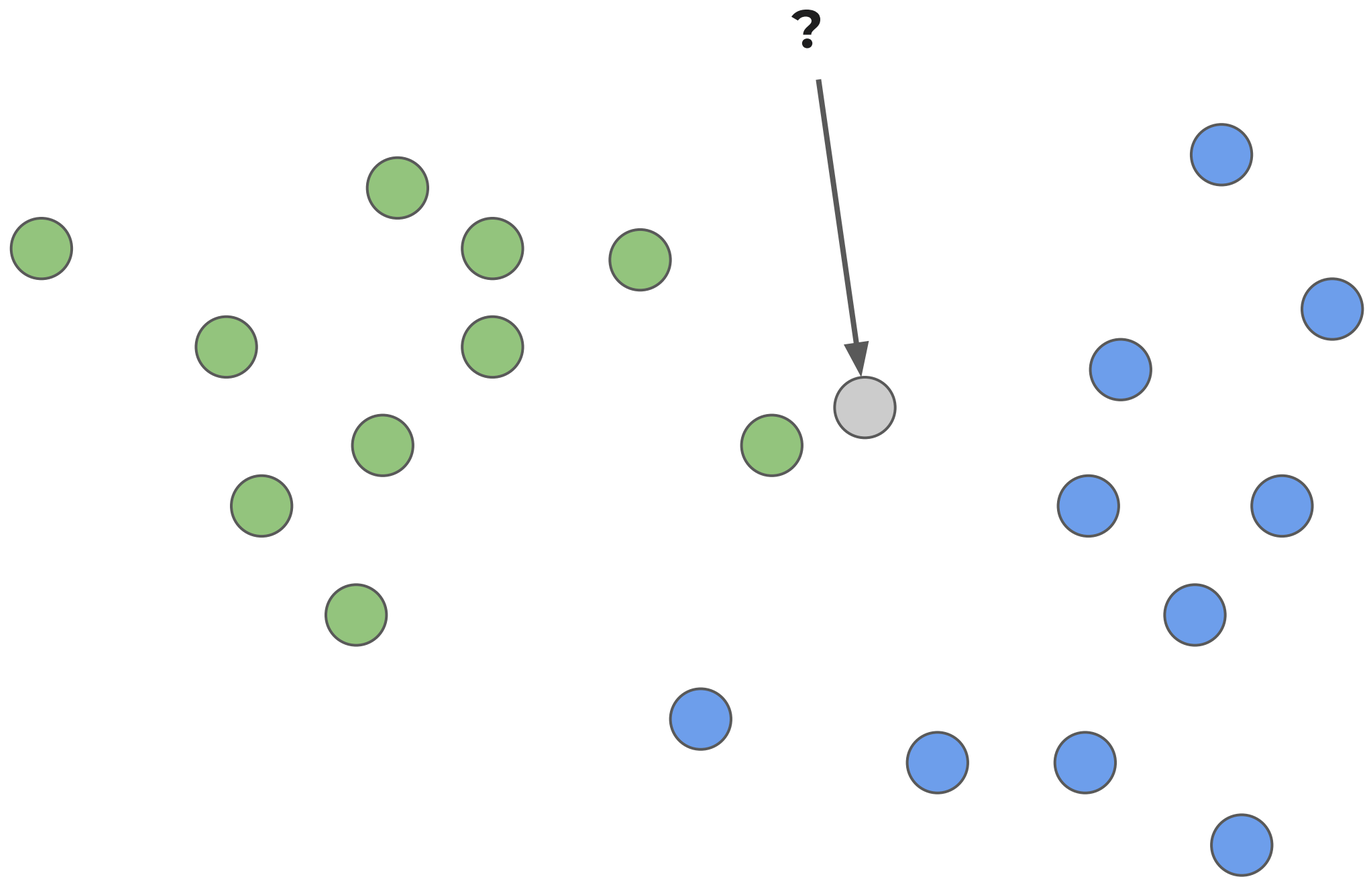
girafe ai

01

Metric based algorithm



Intuition



kNN algorithm

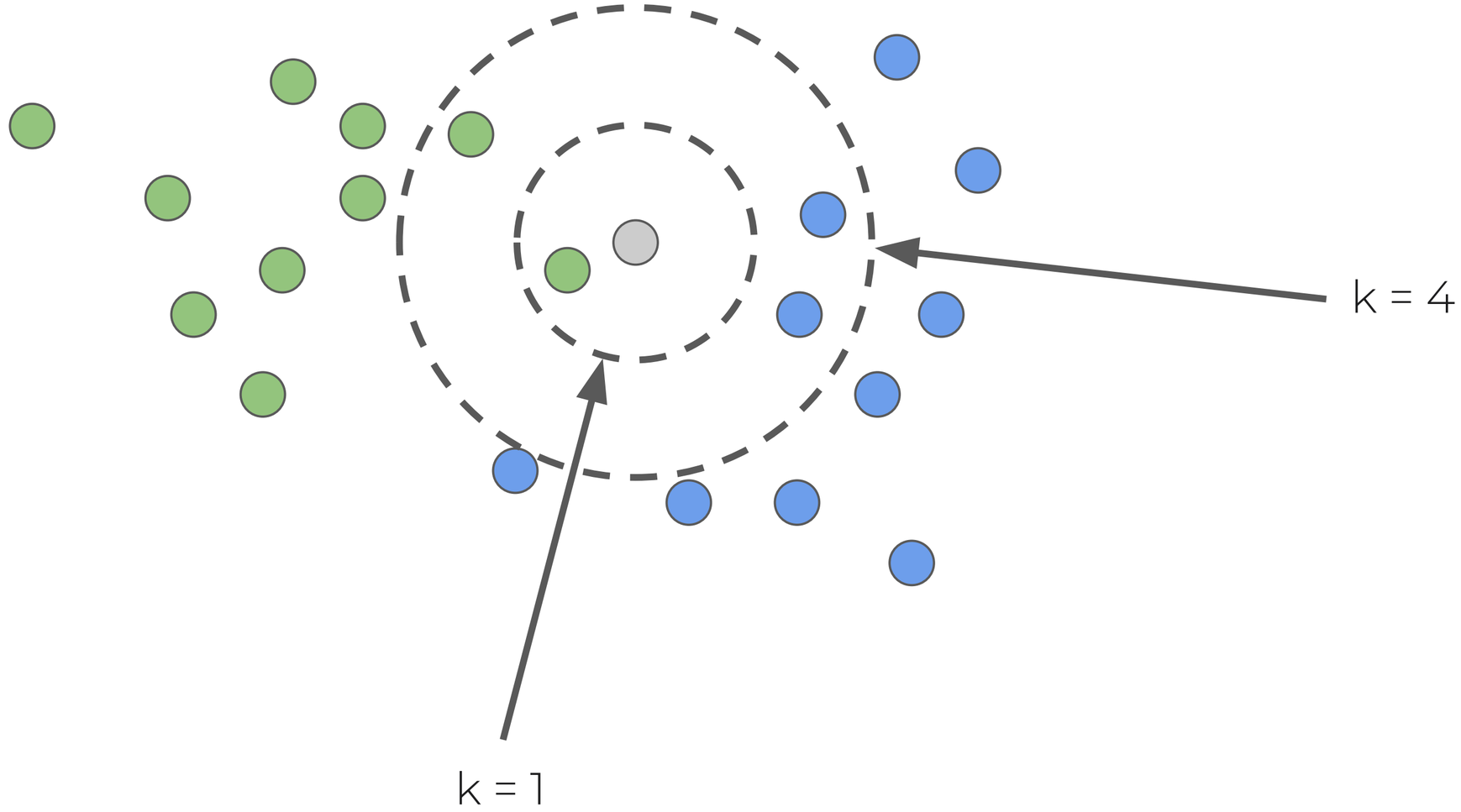
Given a new observation:

1. Calculate the distance to each of the samples in the dataset
2. Select samples from the dataset with the minimal distance to them
3. The label of the new observation will be the most frequent label among those nearest neighbors

How to answer to regression problem?

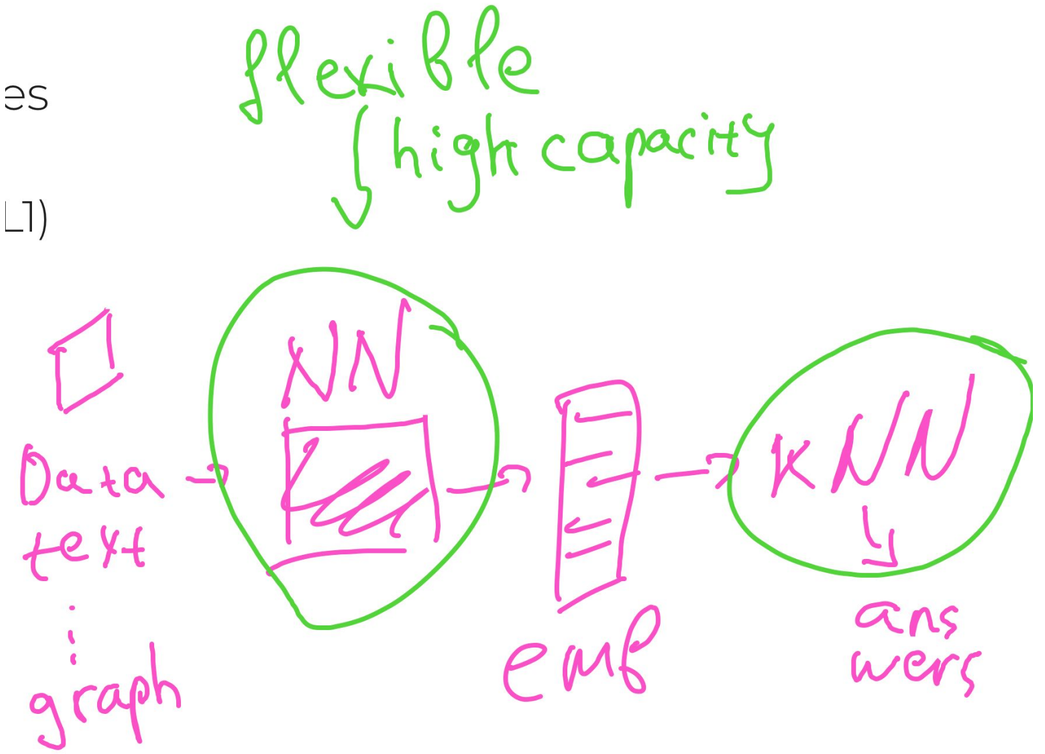
How to make it better?

1. The number of neighbors k



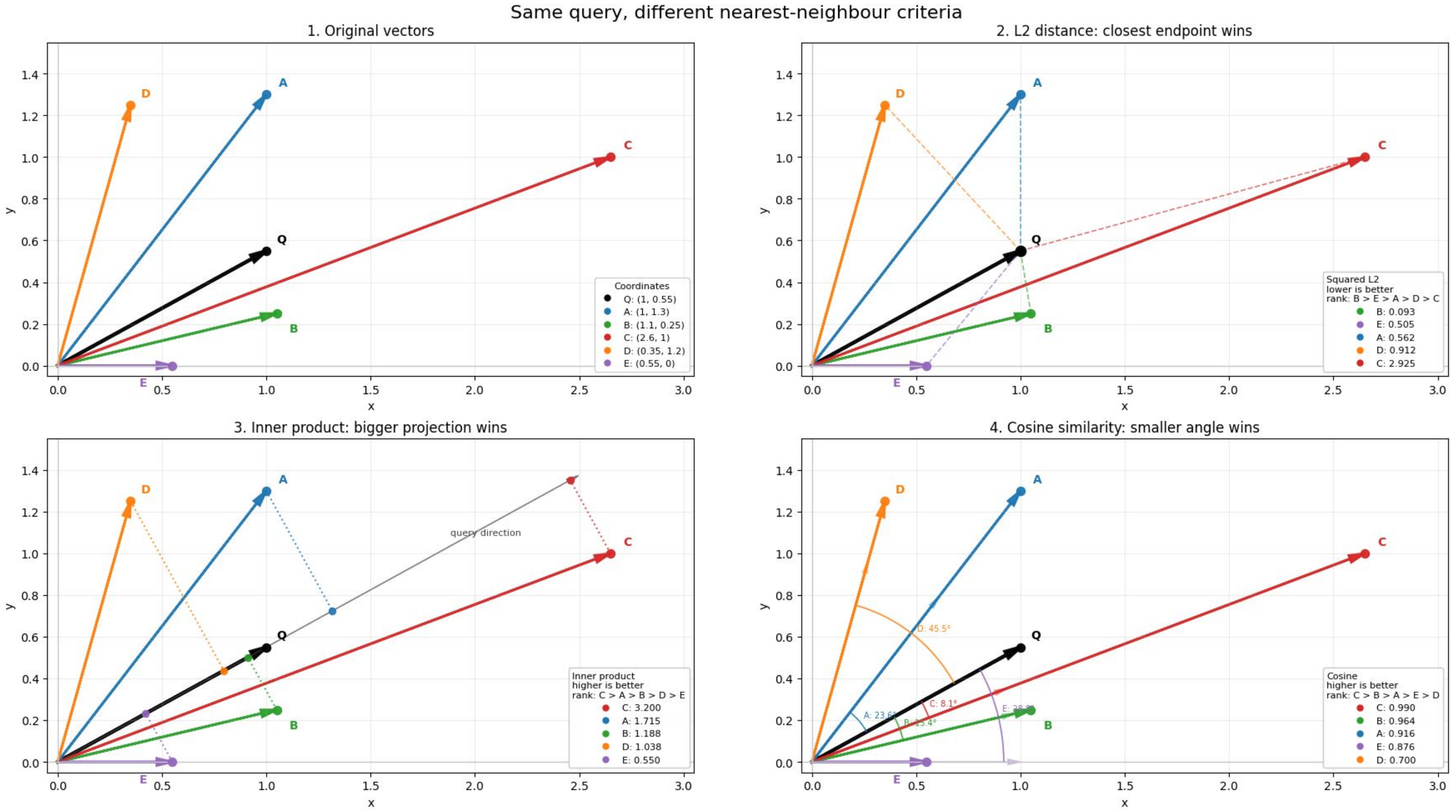
How to make it better?

1. The number of neighbors k
2. The distance measure between samples
   a. Euclidean (L2)
   b. city block distance (aka Manhattan, L1)
   c. Minkowski distances
   d. cosine
   e. Hamming
      i. number of substitutions
   f. etc
3. Weighted neighbours
   a. rarely used in practice



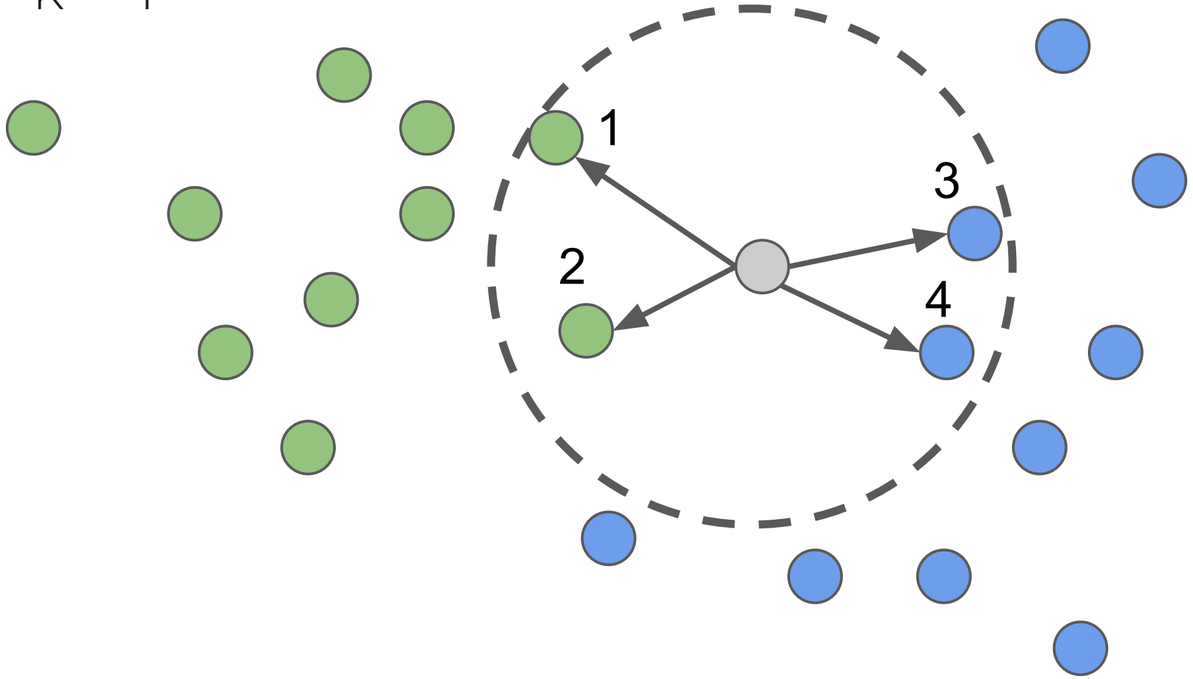
They are hyperparameters for kNN model

Distances in kNN



Weighted kNN

k = 4



- Weights can be adjusted according to the neighbors order

$$w(\mathbf{x}_{(i)}) = w_i$$

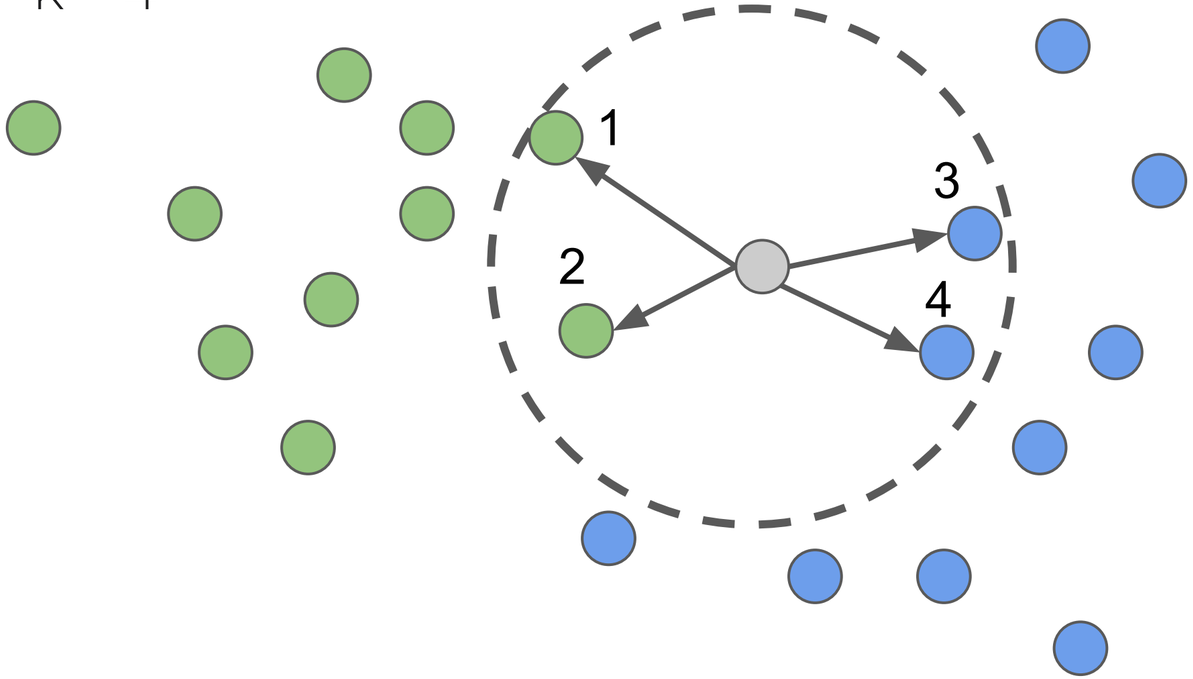
- or on the distance itself

$$w(\mathbf{x}_{(i)}) = w(d(\mathbf{x}, \mathbf{x}_{(i)}))$$

$$p_{\text{green}} = \frac{w(\mathbf{x}_1) + w(\mathbf{x}_2)}{w(\mathbf{x}_1) + w(\mathbf{x}_2) + w(\mathbf{x}_3) + w(\mathbf{x}_4)}$$

Weighted kNN

k = 4



- Weights can be adjusted according to the neighbors order

$$w(\mathbf{x}_{(i)}) = w_i$$

- or on the distance itself

$$w(\mathbf{x}_{(i)}) = w(d(\mathbf{x}, \mathbf{x}_{(i)}))$$

$$p_{\text{blue}} = \frac{w(\mathbf{x}_3) + w(\mathbf{x}_4)}{w(\mathbf{x}_1) + w(\mathbf{x}_2) + w(\mathbf{x}_3) + w(\mathbf{x}_4)}$$

Parameters and hyperparameters

Parameters:

none. This is non-parametric model.

Hyperparameters:

- k - number of neighbours
- distance function
  - euclidean, cosine distance
- others are used very rarely

Naive compute approach

Common object for kNN nowadays is embedding (usually by neural networks).

Let’s suppose we have **million (10^6)** of floating point **embeddings in 100 dimensional** space.

We have a computer with **1 GHz (10^9) operations** frequency.

Then we want to find **10 nearest neighbours** of a new given embedding.

How much time will it take?

Is this time acceptable for realtime operation?

kNN indexes

girafe
ai

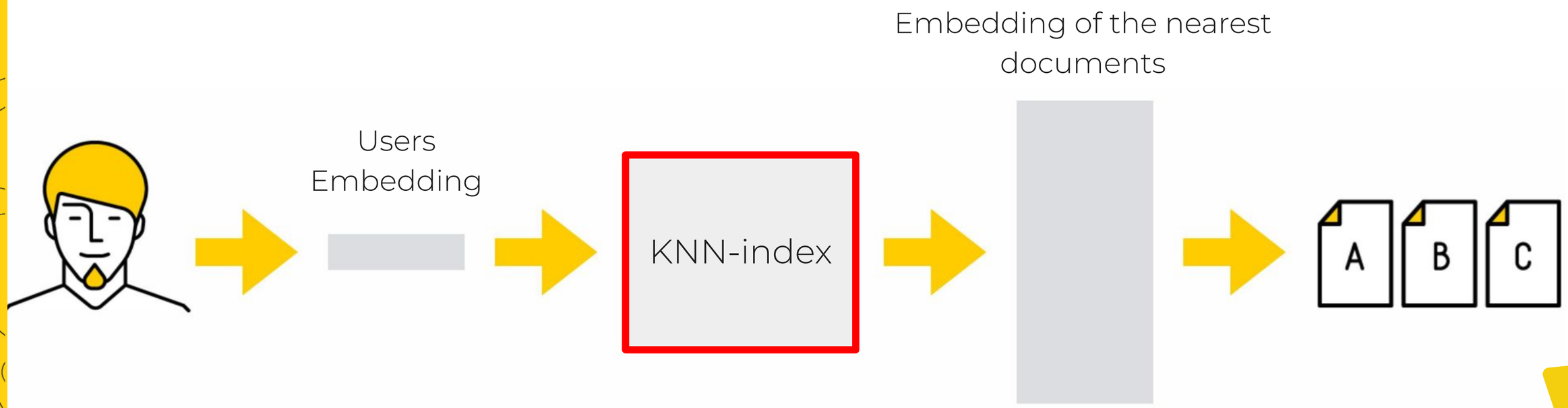
02

kNN index

KNN index is a data structure
that allows you to quickly find the N nearest vectors for a given vector.

We perform some precompute beforehand to increase speed of final search.

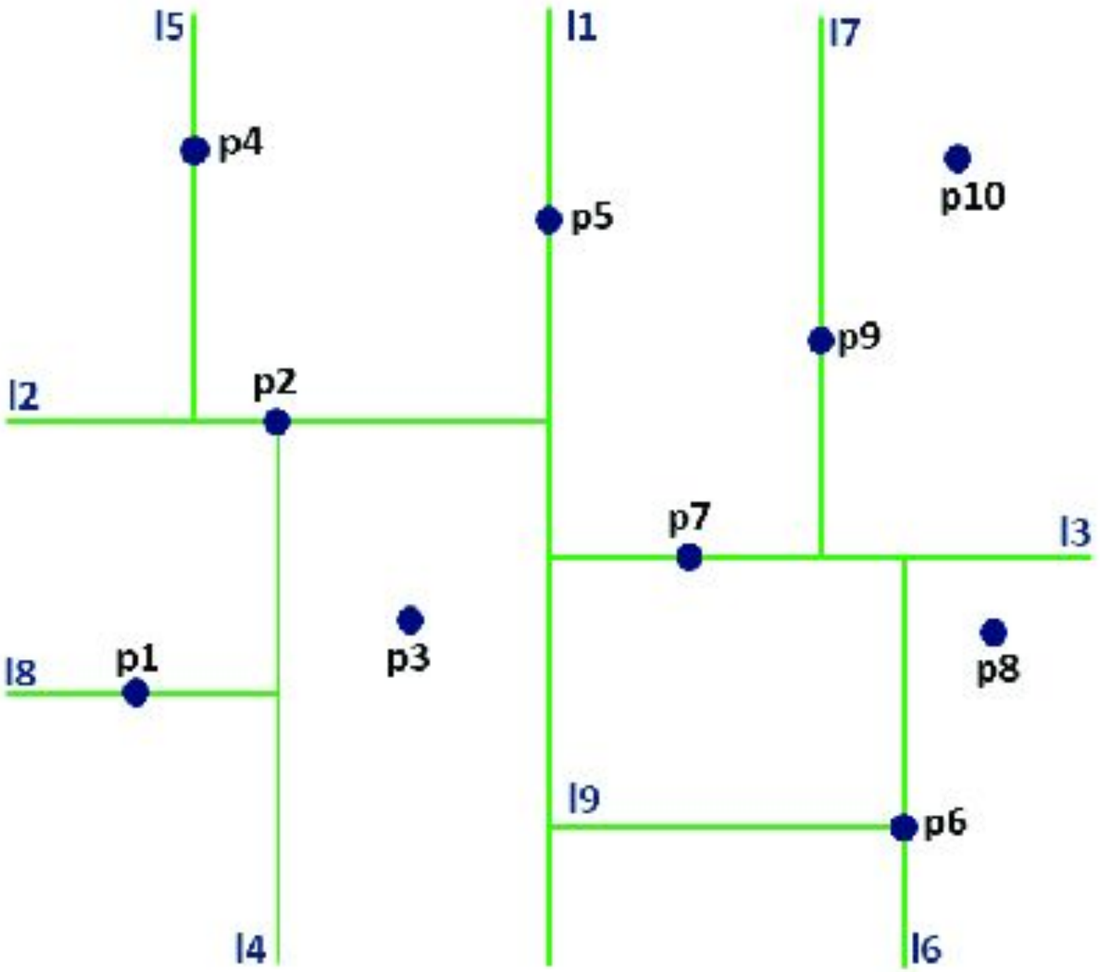
This approach is also called **Approximate Nearest Neighbor (ANN)**

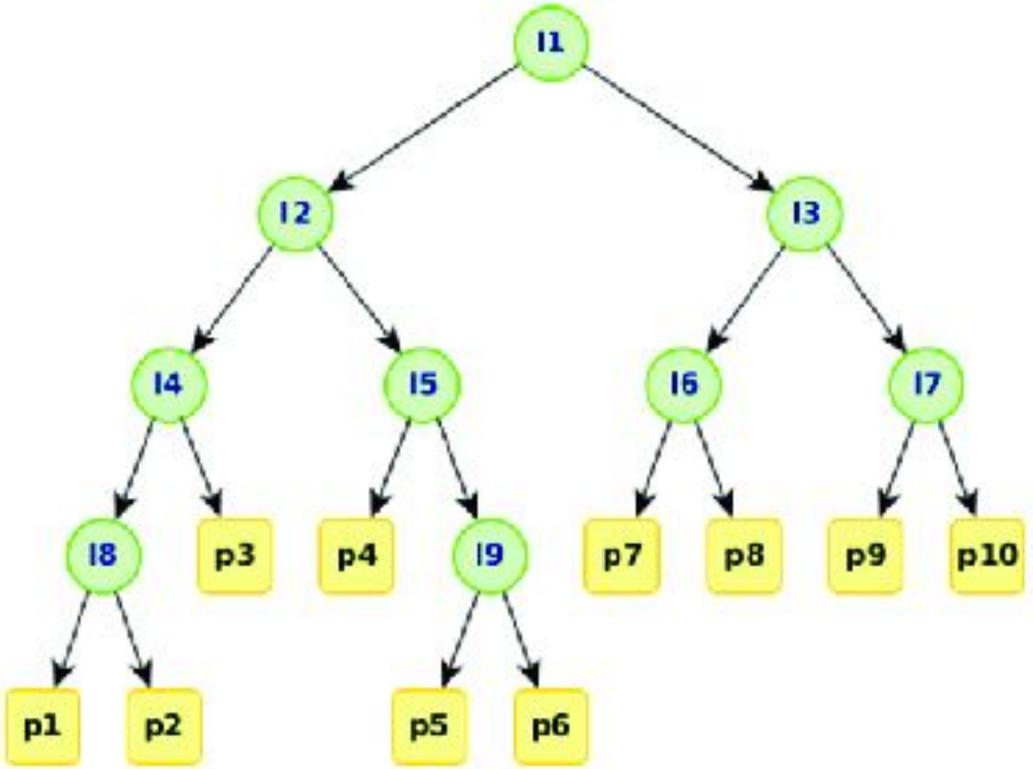


kNN index ideas

1. Dimensionality reduction
    a. Quantization
2. Clustering
3. Trees
    a. k-d tree
        i. default for scikit learn
        ii. ball tree is extension for high dimensional cases
4. Grid / Graph
    a. HNSW
        i. most common nowadays
5. Hashing
    a. Locally sensitive
6. many more...

k-d tree


l5
l1
l7
p4
p5
p10
p9
l2
p2
p7
l3
p3
p8
l8
p1
l9
p6
l4
l6


l1
l2
l3
l4
l5
l6
l7
l8
l9
p3
p4
p7
p8
p9
p10
p1
p2
p5
p6

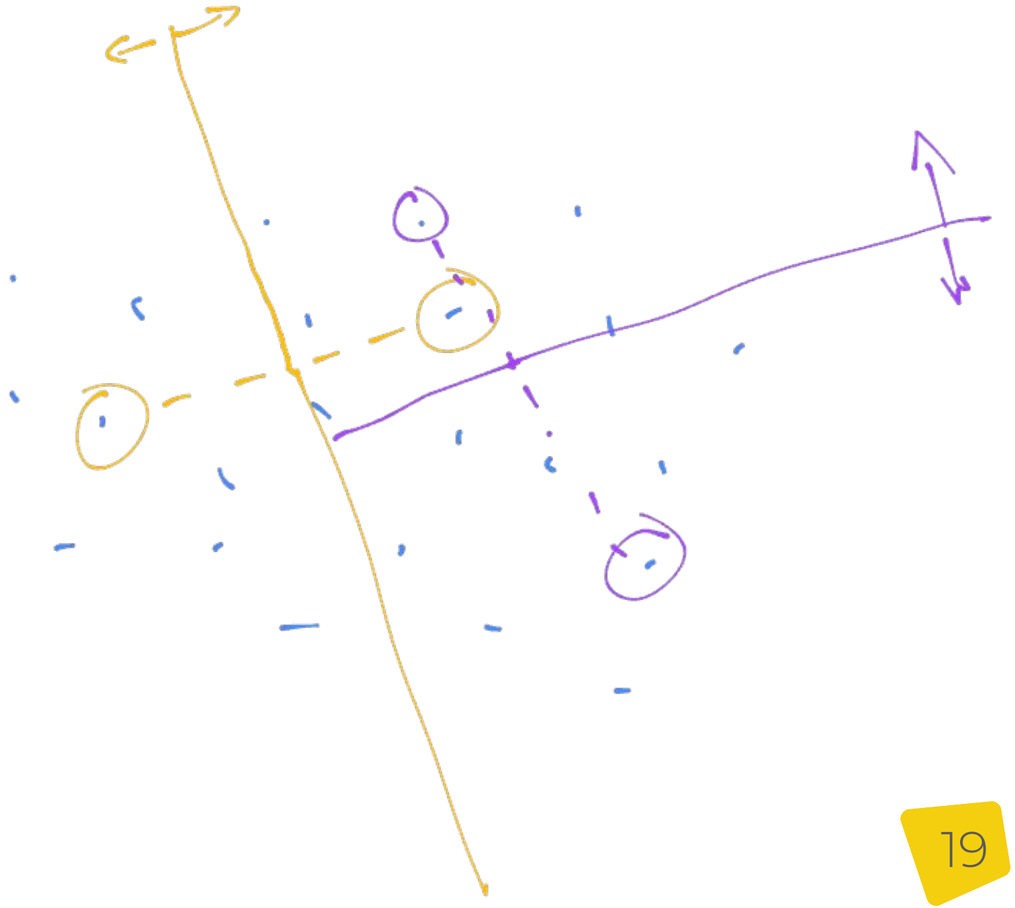
Approximate Nearest Neighbor Oh Yeah (ANNOY) by Spotify

1. take two random points
2. build a hyperplane, normal to the segment
3. split dataset with this hyperplane

Repeat while you have enough points

https://sds-aau.github.io/M3Port19/portfolio/ann/

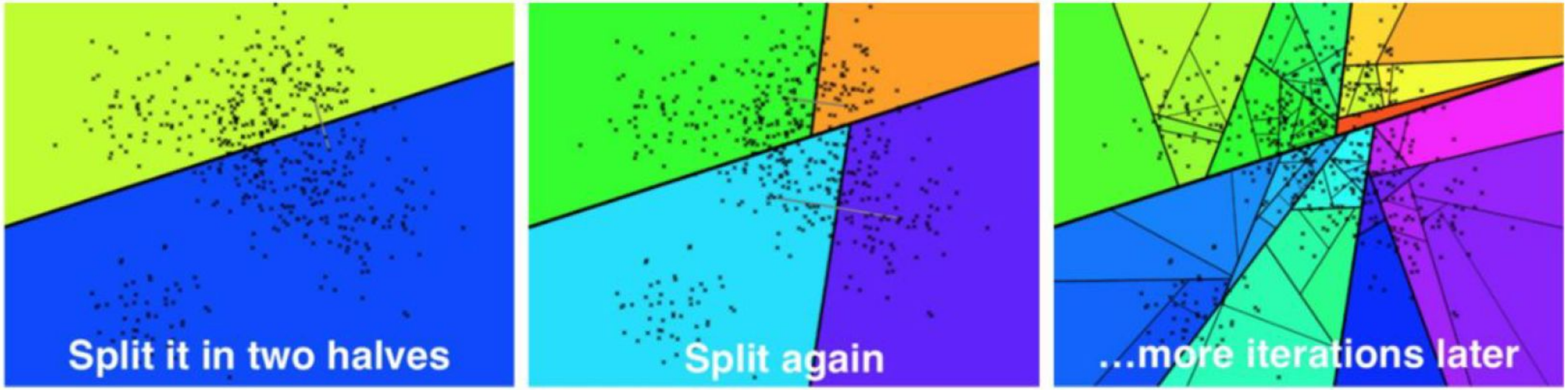
https://github.com/spotify/annoy



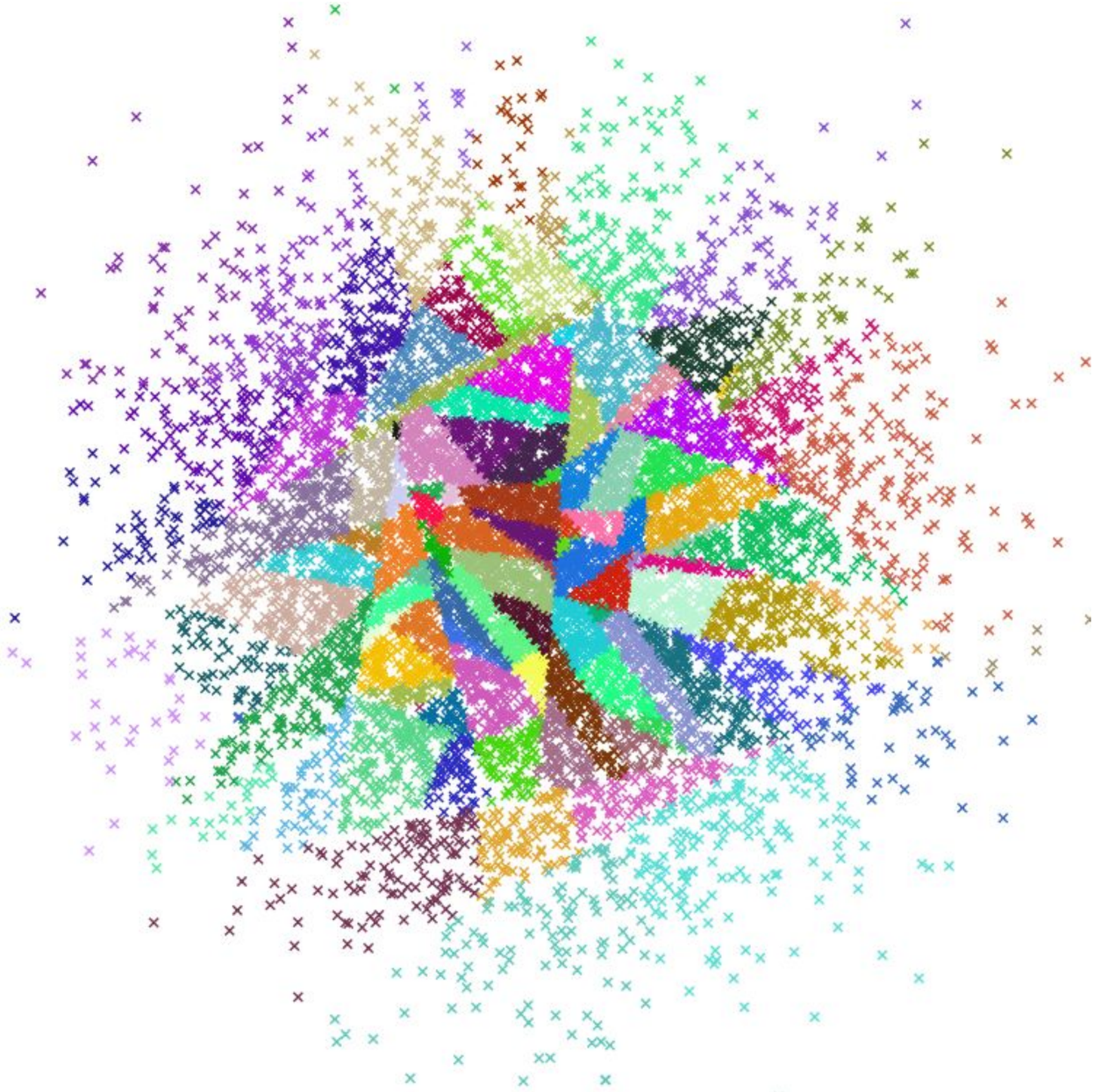
ANNOY example

One tree gives approximate result.

A forest gives much better answer.



ANNOY example



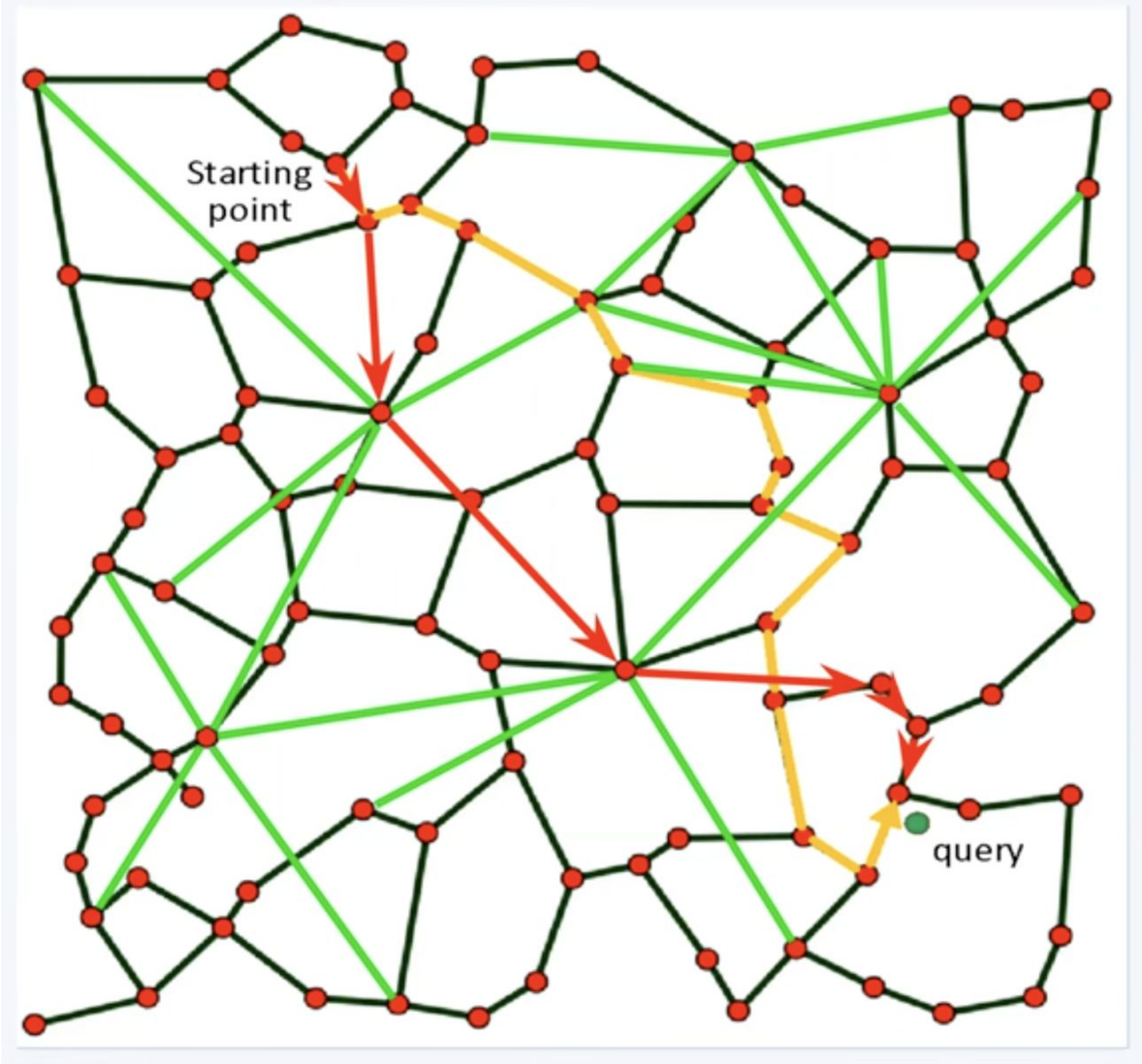
Navigable Small World (NSW)

girafe ai

03

NSW (Navigable Small World)

https://www.youtube.com/watch?v=4PsyNdFIxmk



1. Connect nearest neighbours
2. Add long connections

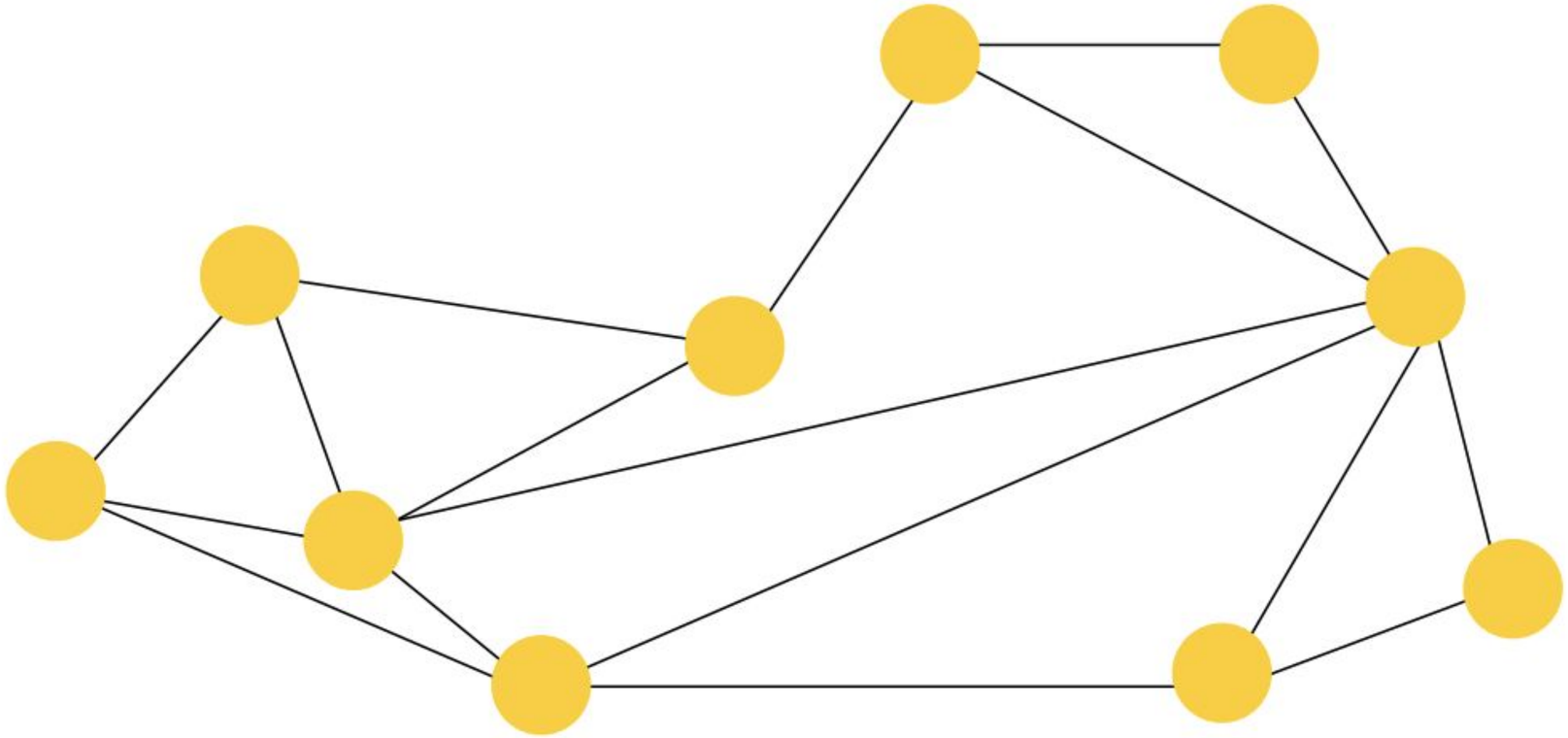
Navigable Small World



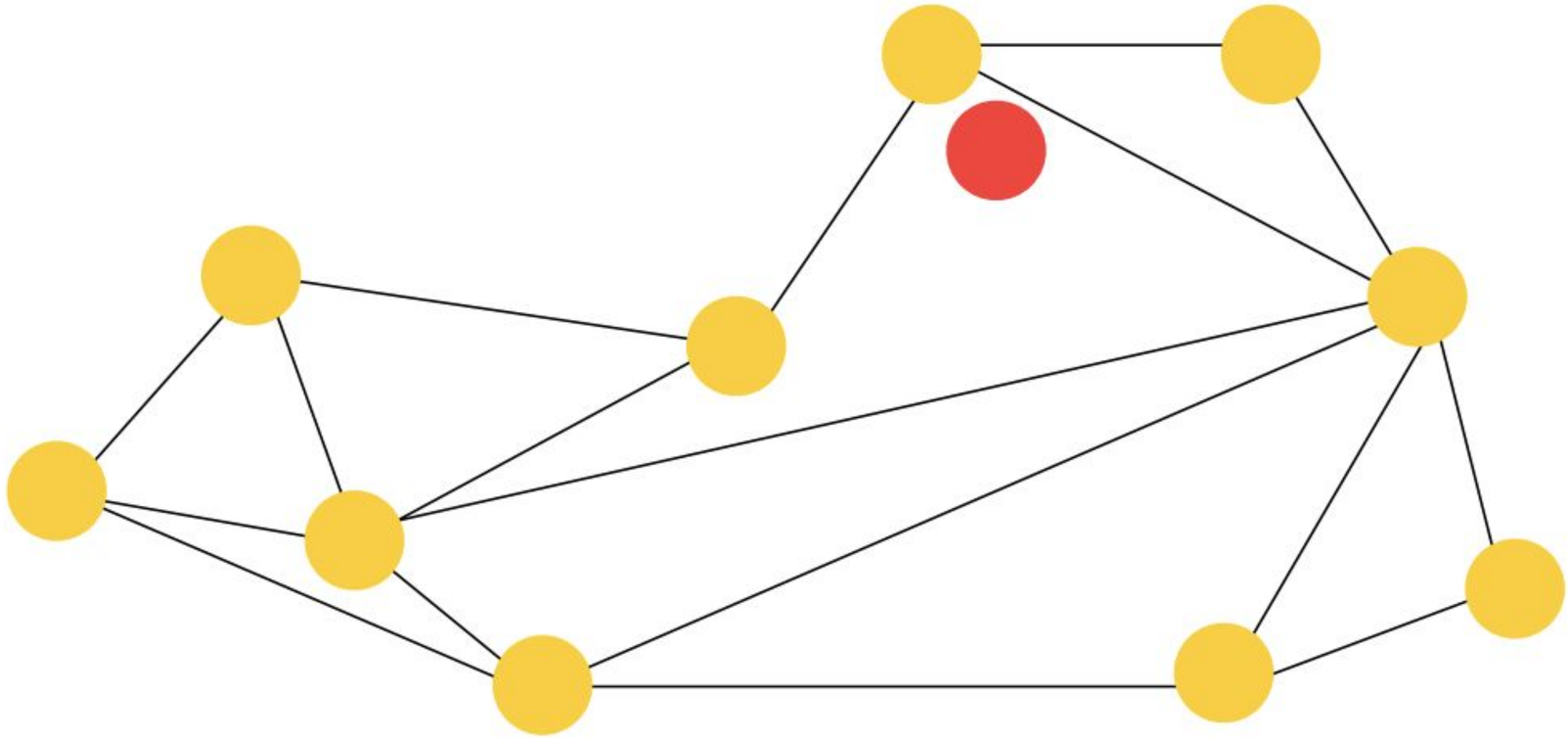
- A graph where vertices are embeddings, edges are distances
- Adding nearby and remote vertices as edges
- We start from a random vertex, count the distances and move along the graph closer to the query point

https://arxiv.org/abs/1603.09320

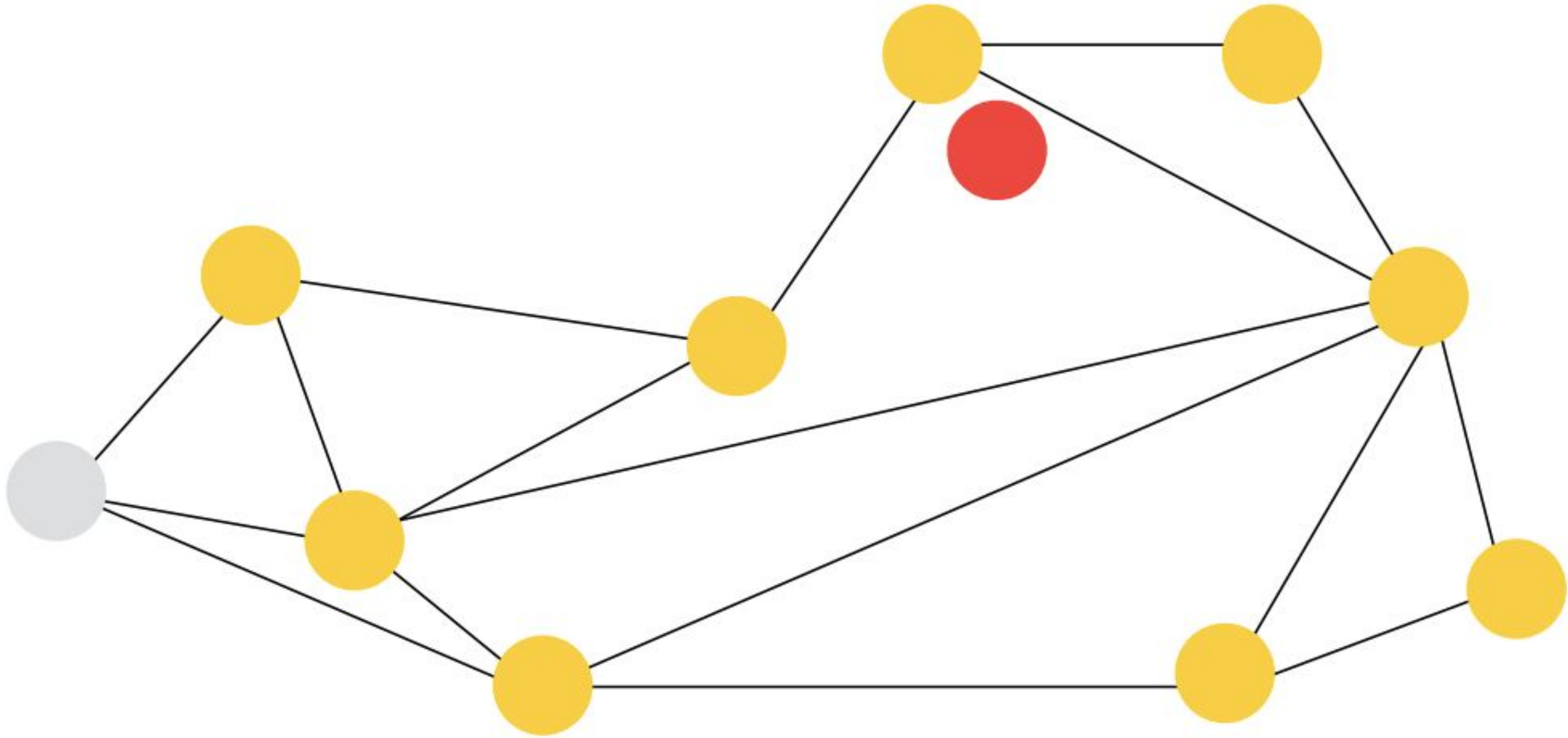
NSW (Navigable Small World)



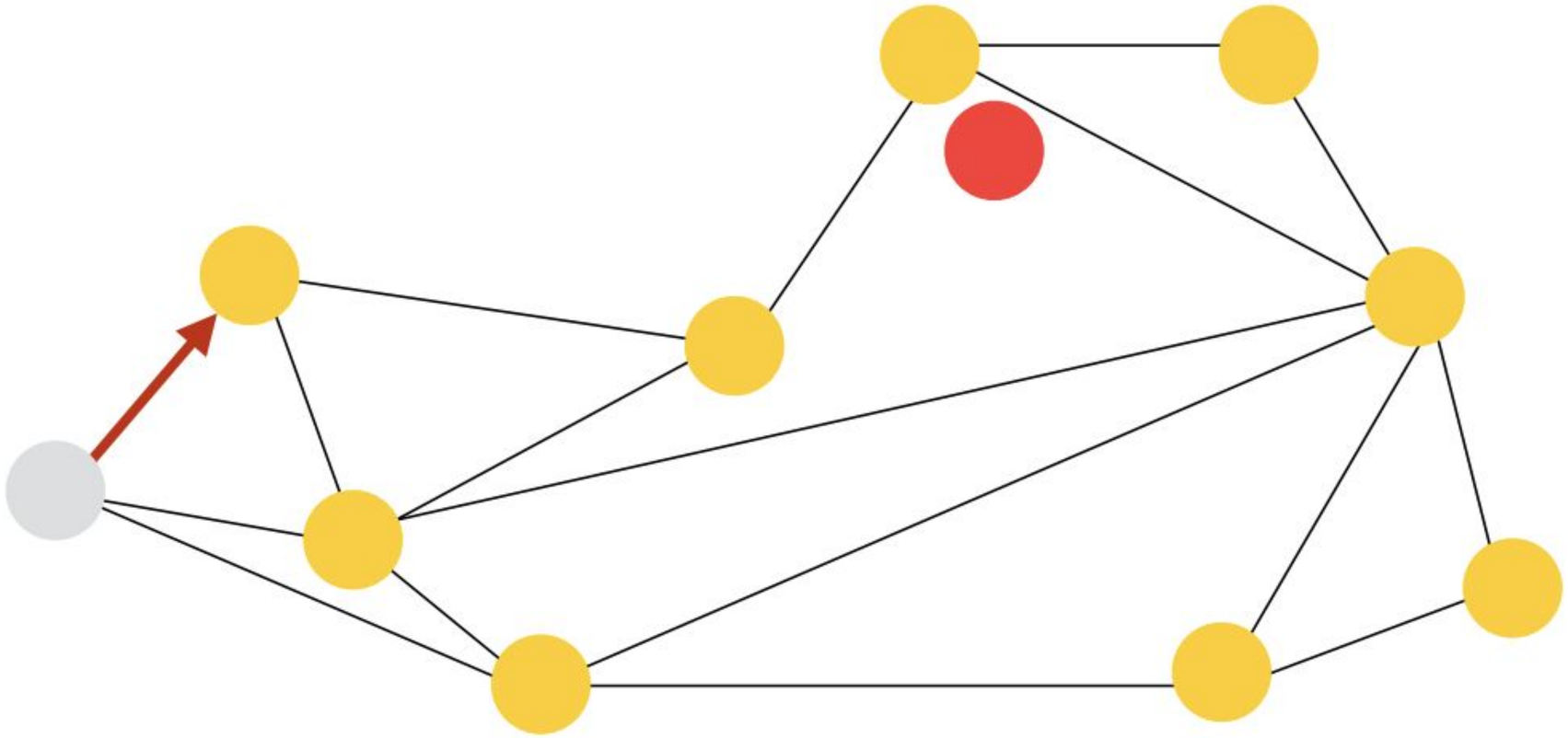
NSW (Navigable Small World)



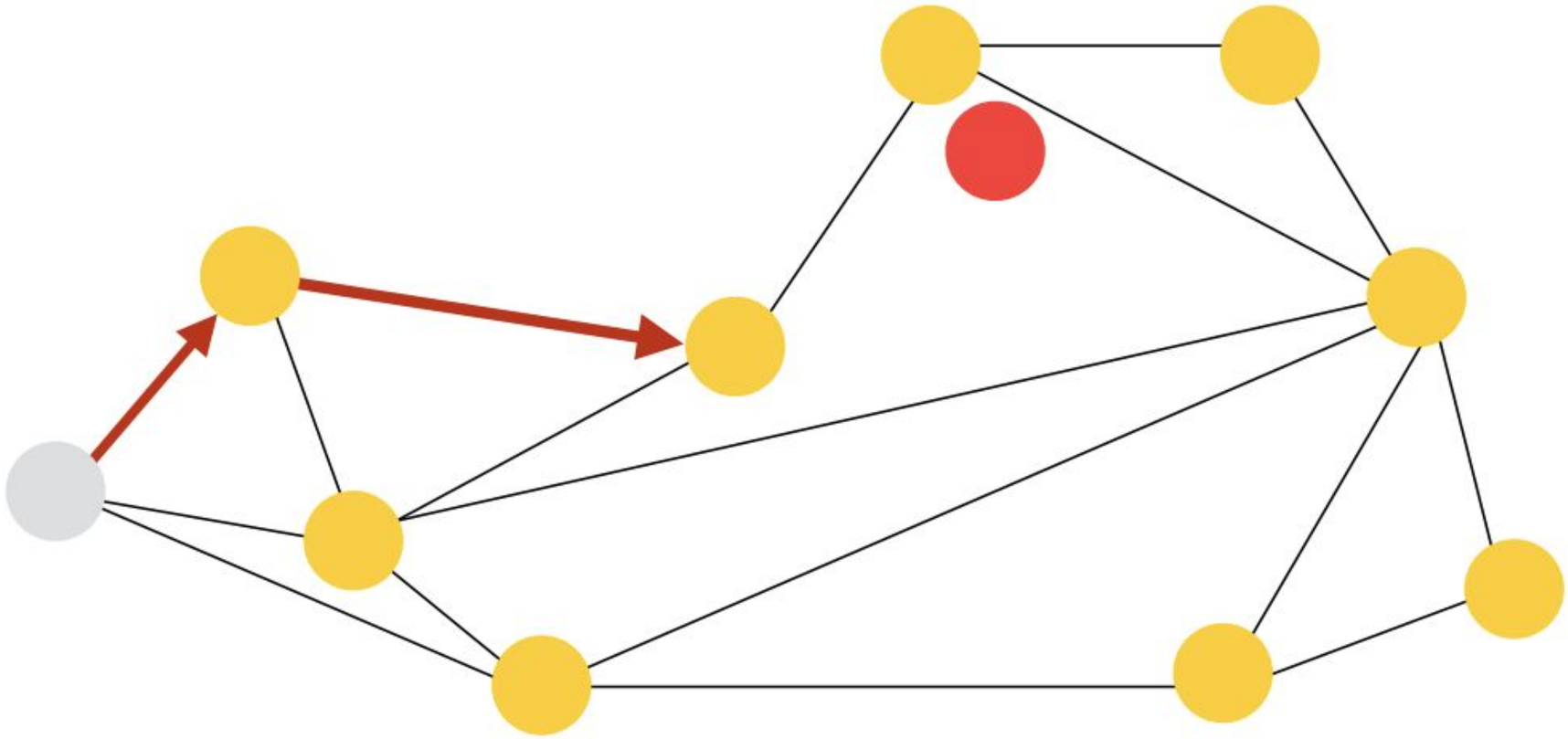
NSW (Navigable Small World)



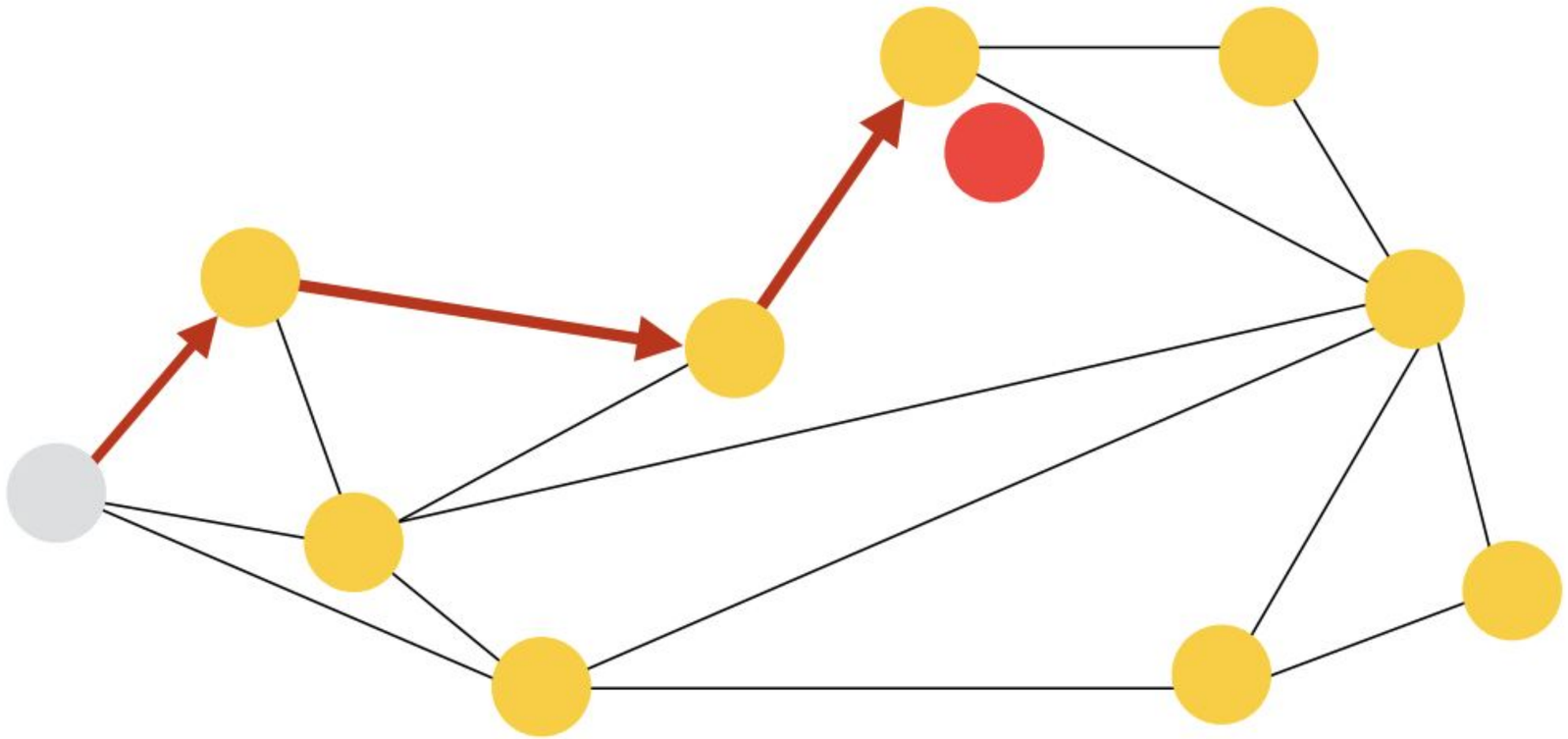
NSW (Navigable Small World)



NSW (Navigable Small World)



NSW (Navigable Small World)



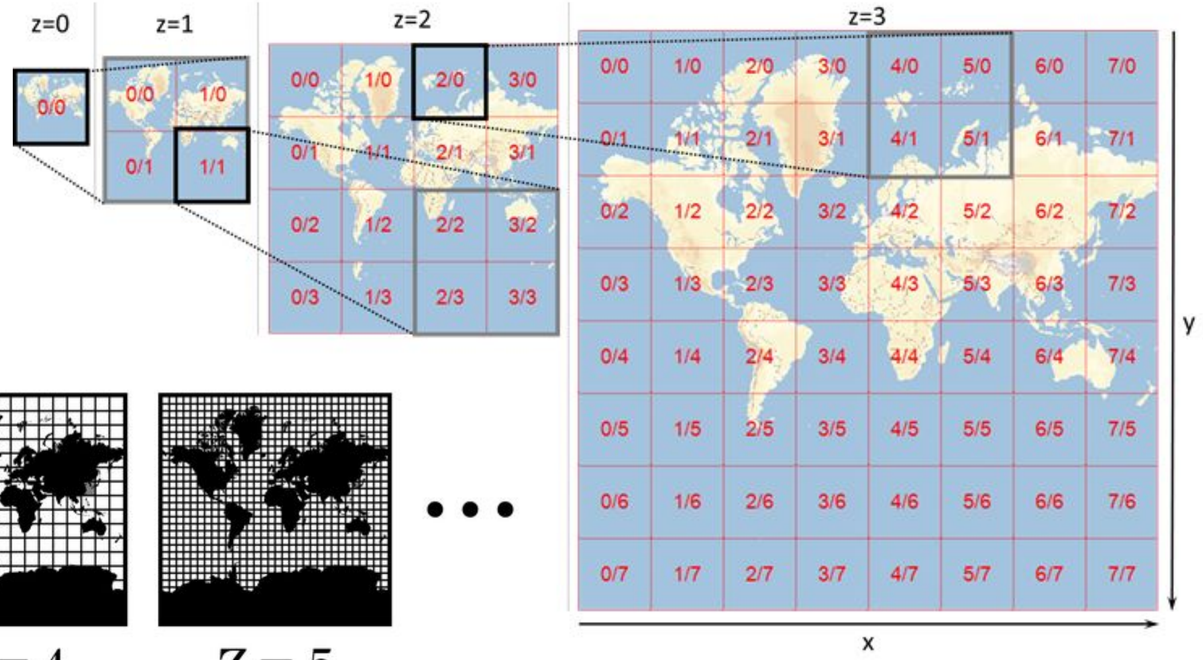
Hierarchical Navigable Small World (HNSW)

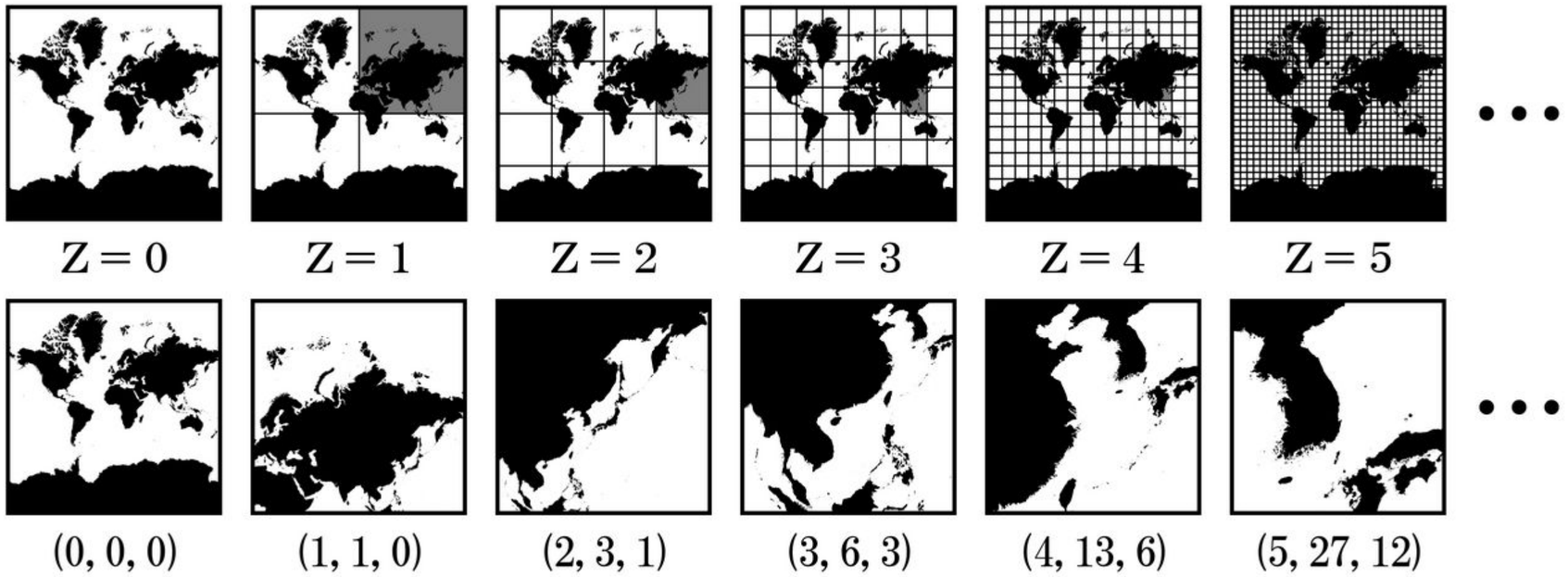
girafe ai

04

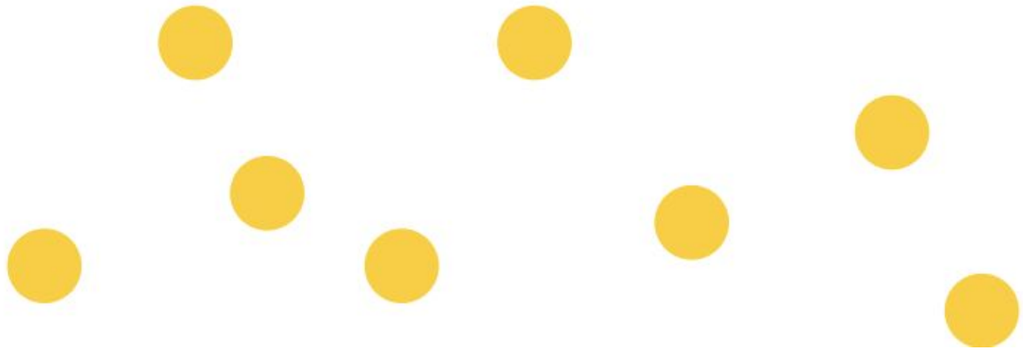
Idea from web tiled maps

https://en.wikipedia.org/wiki/Tiled_web_map

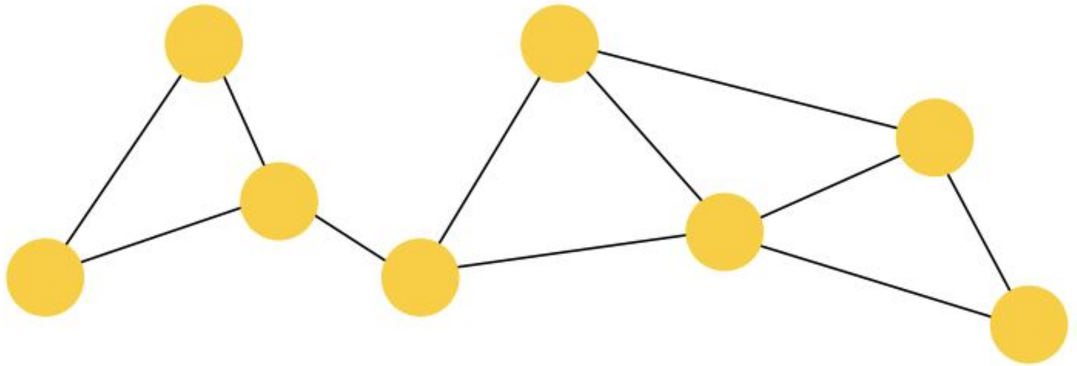




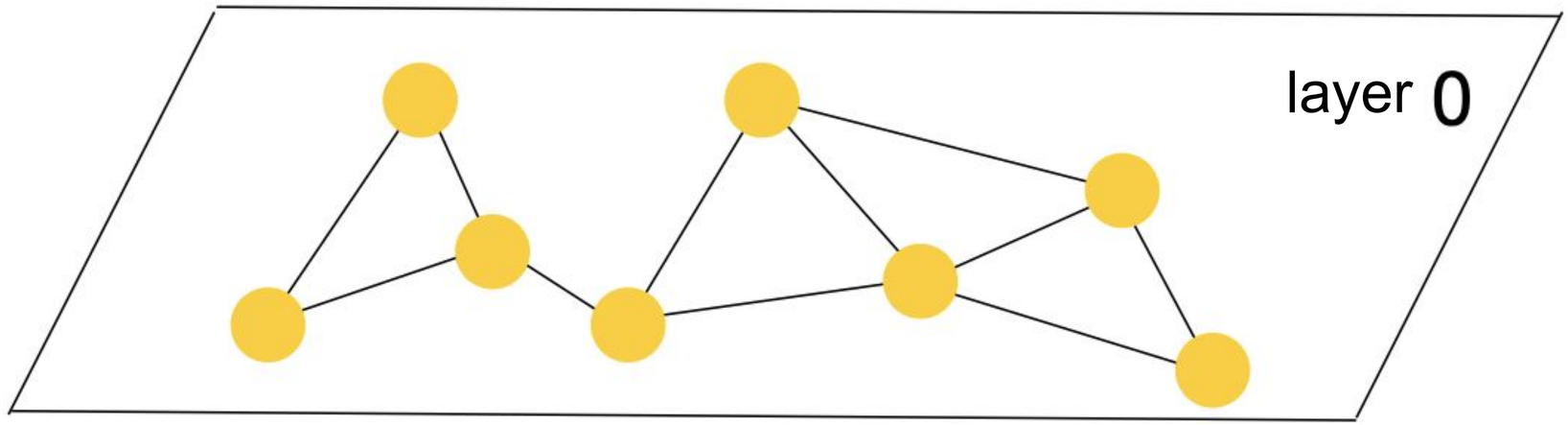
HNSW



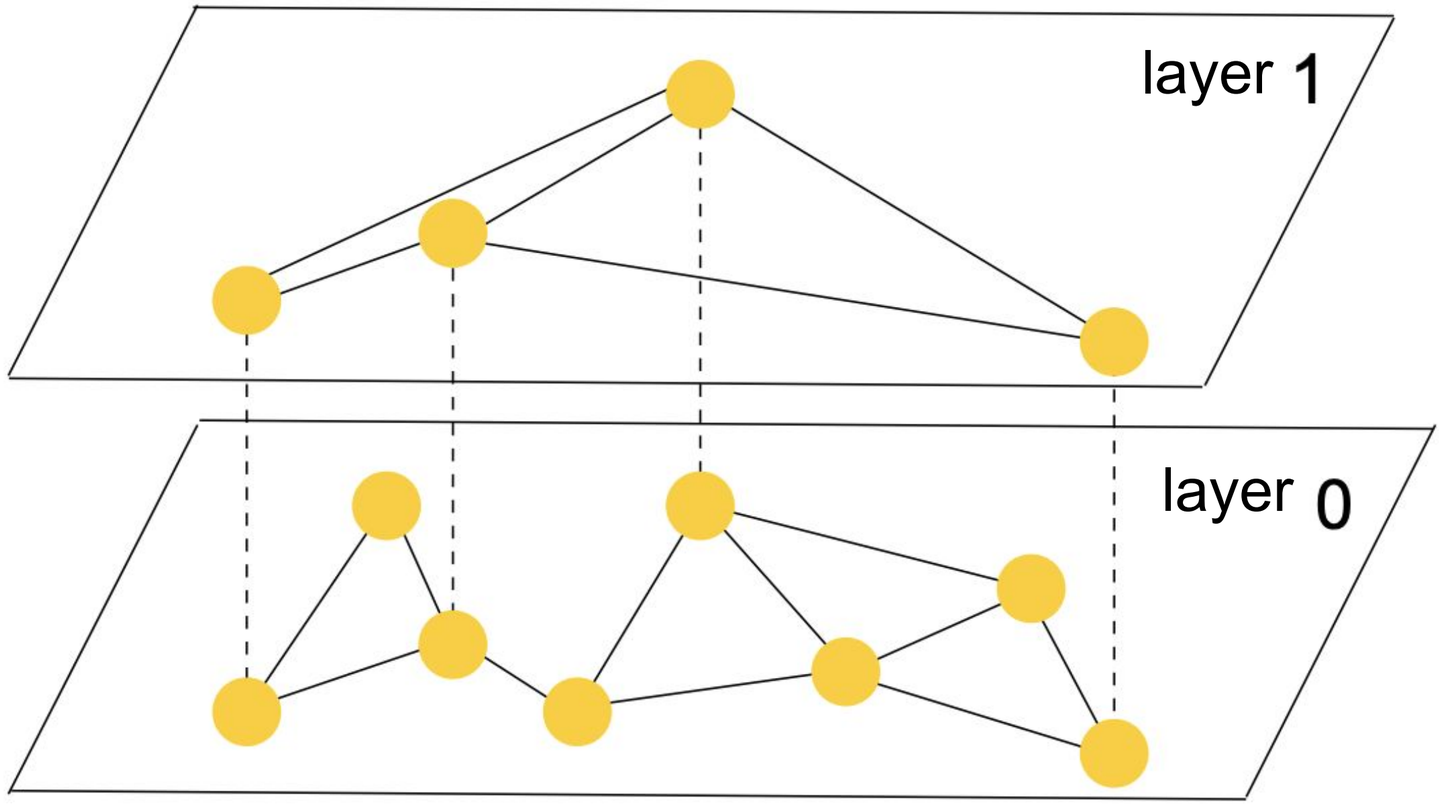
HNSW



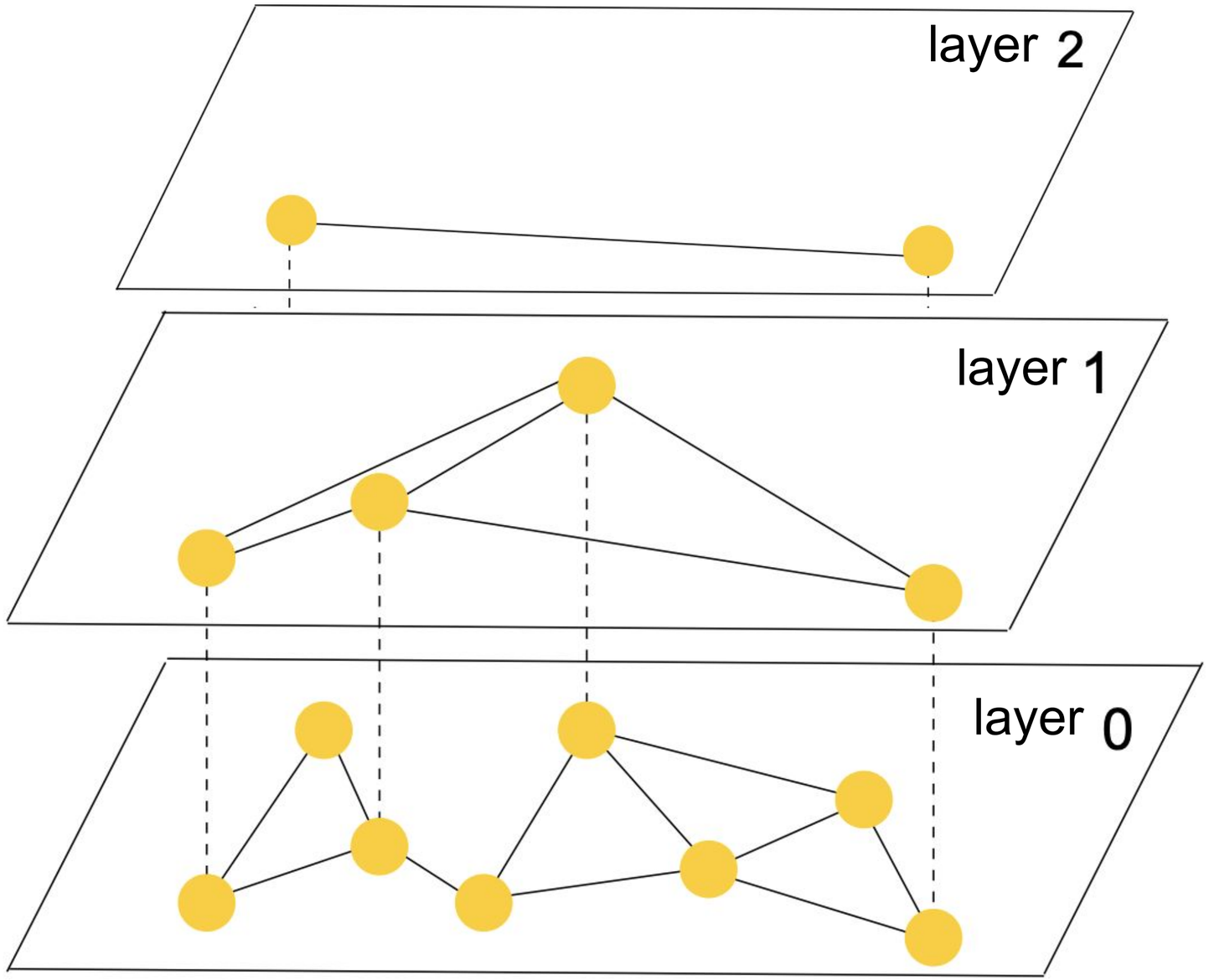
HNSW



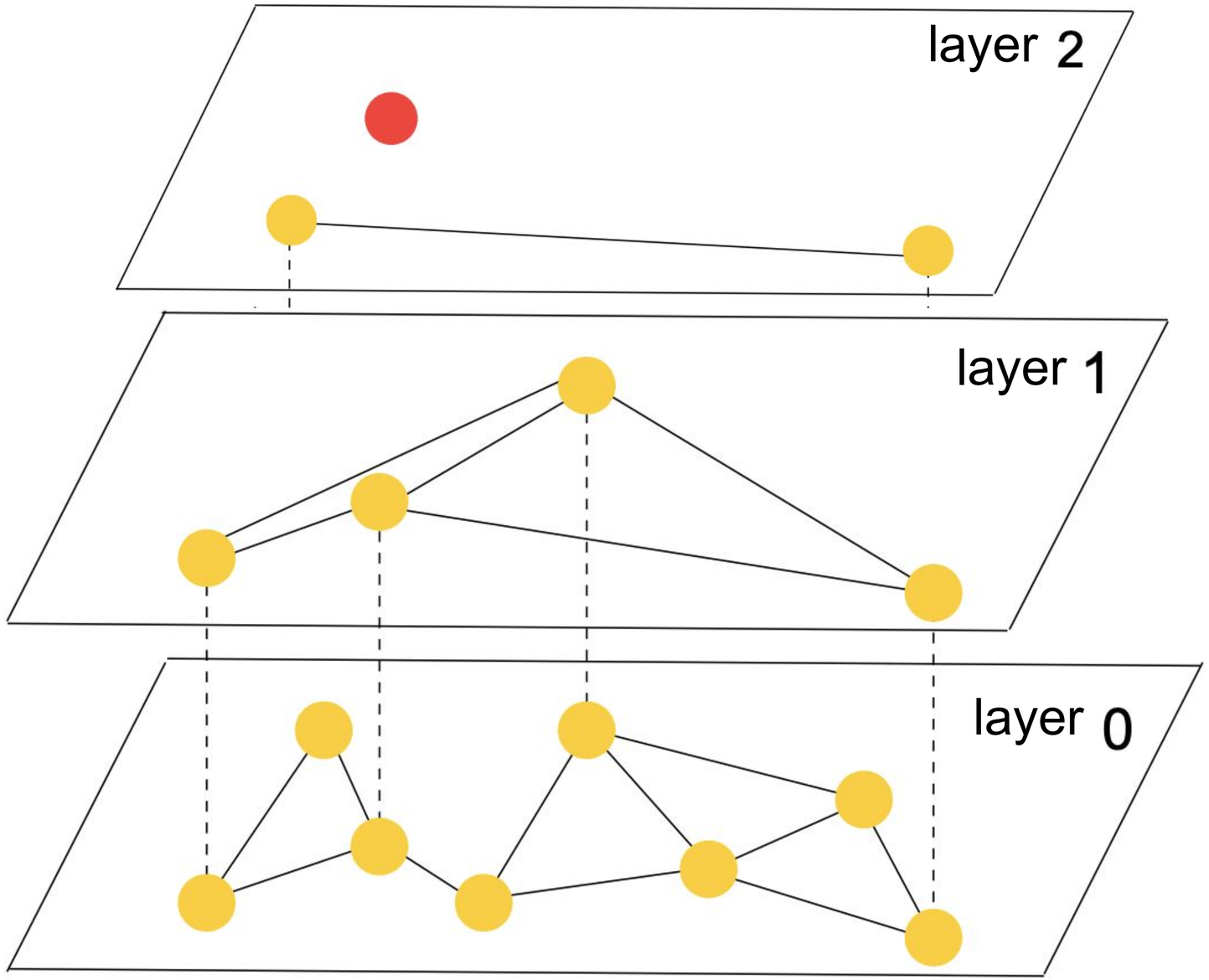
HNSW



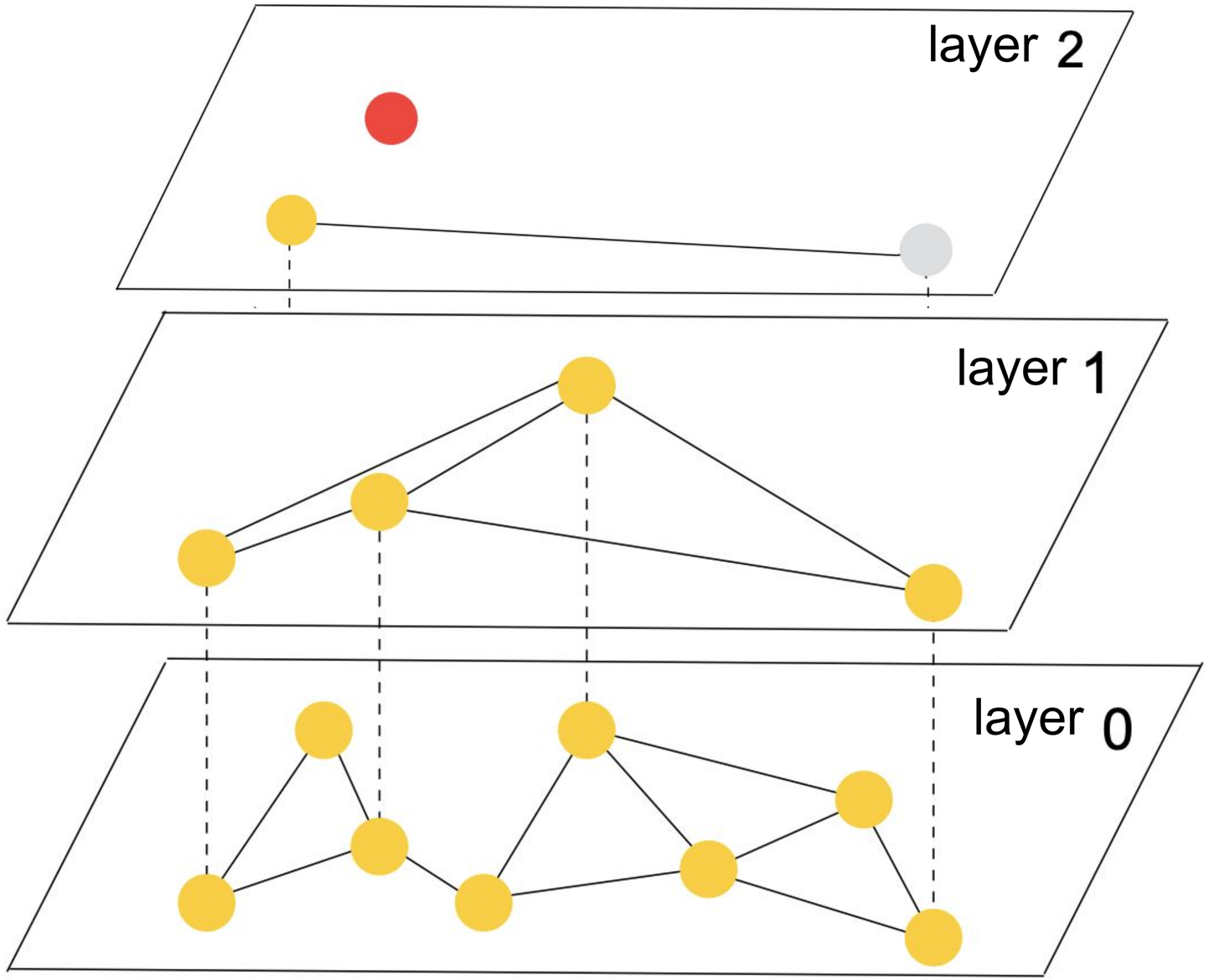
HNSW



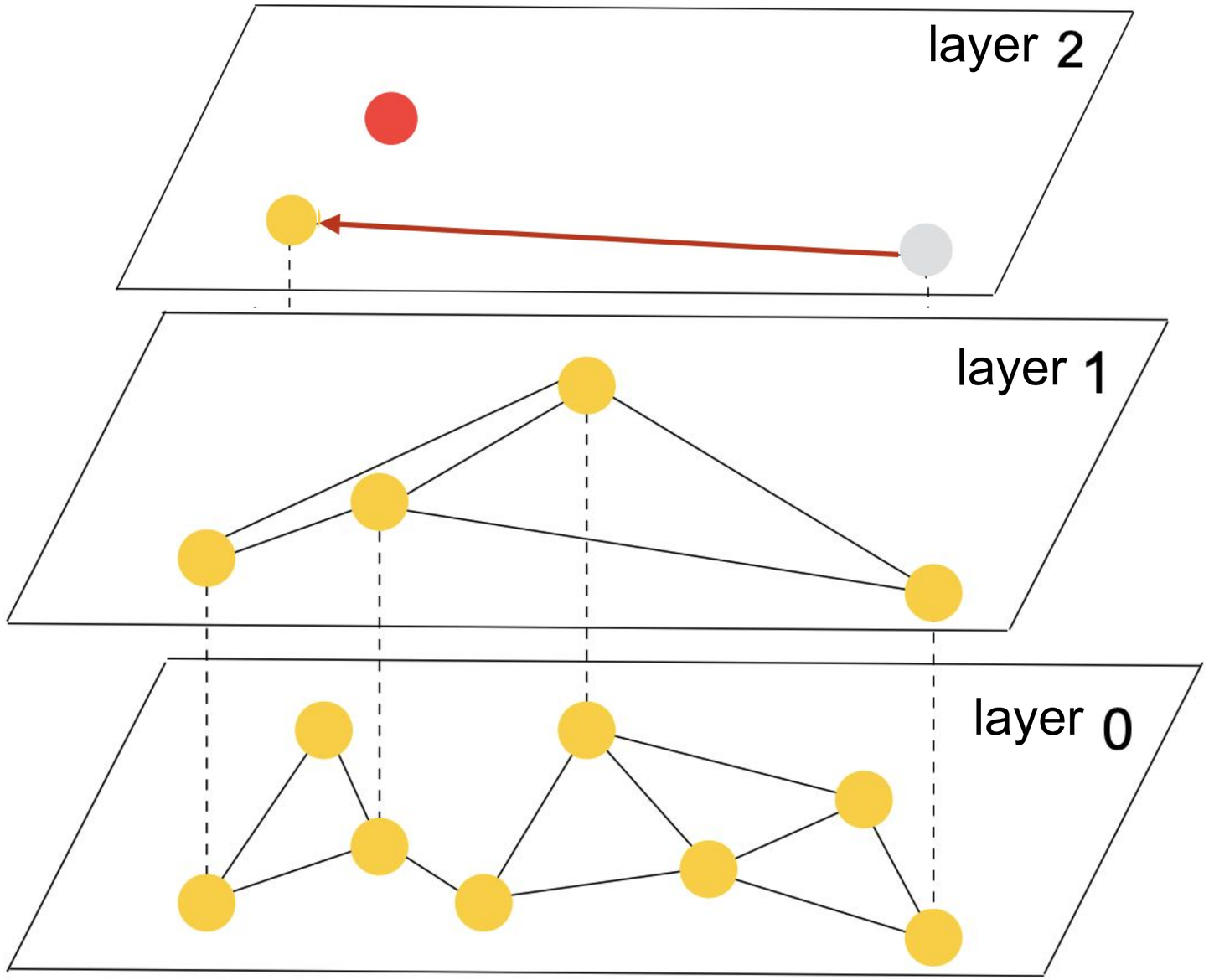
HNSW



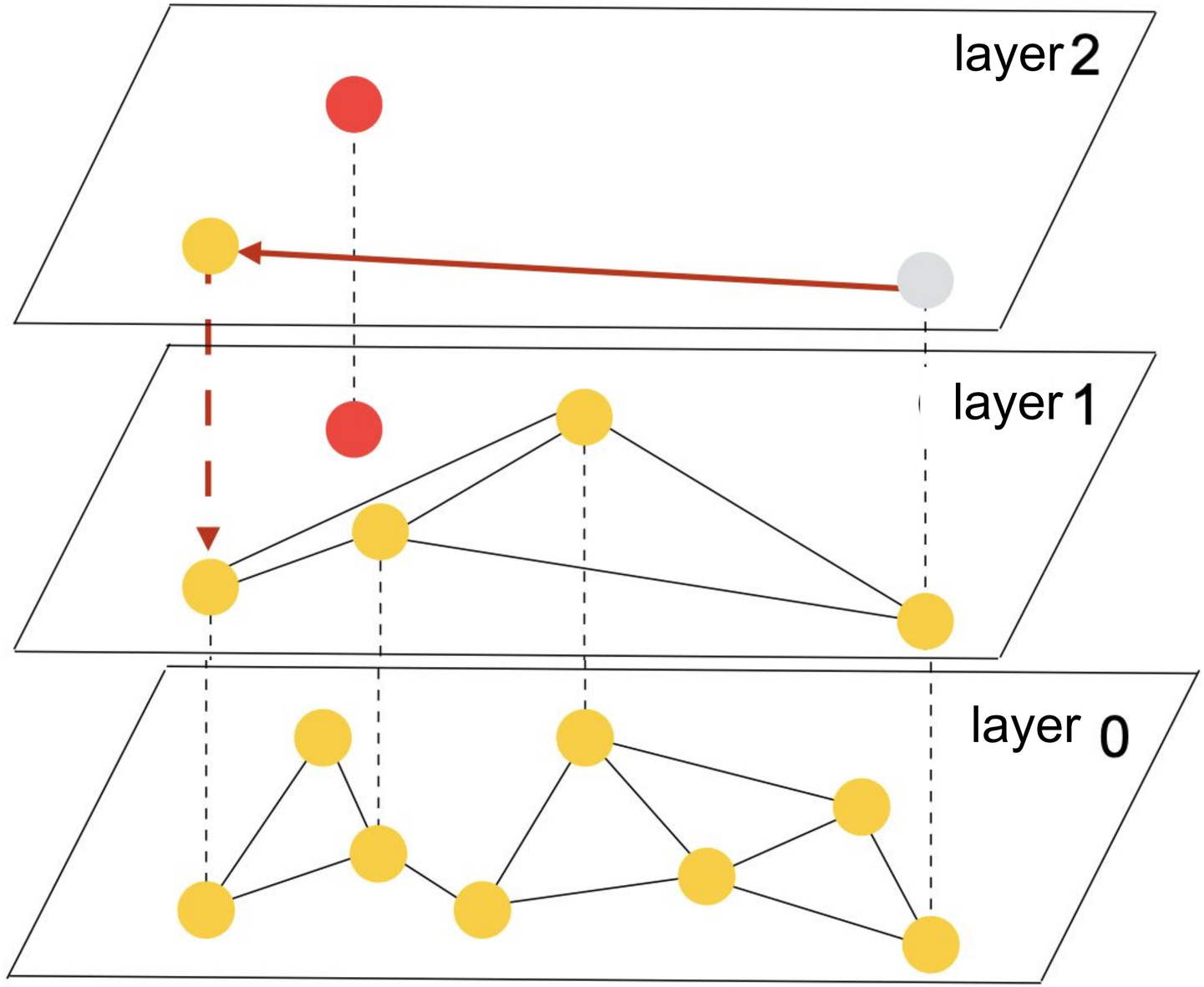
HNSW



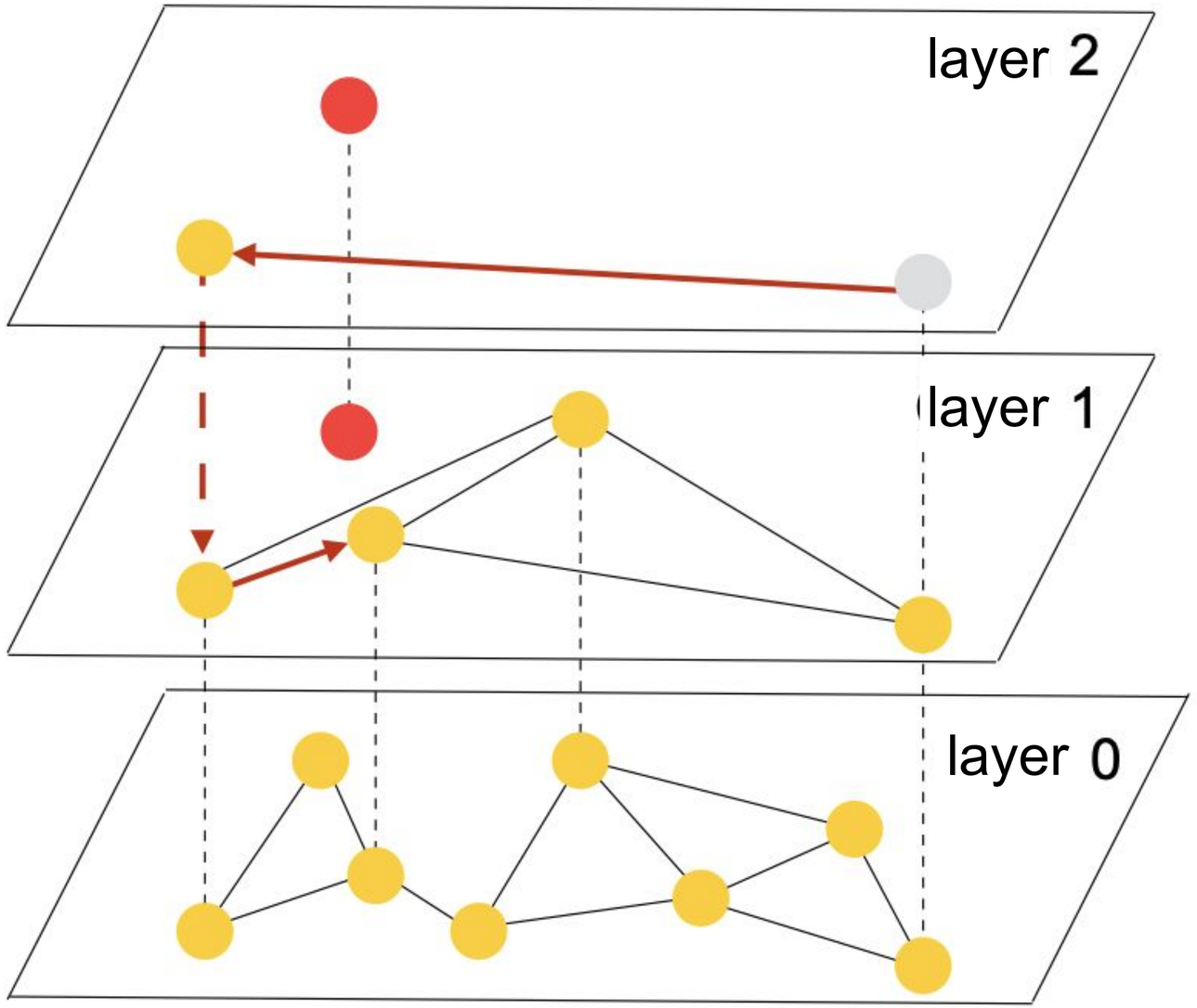
HNSW



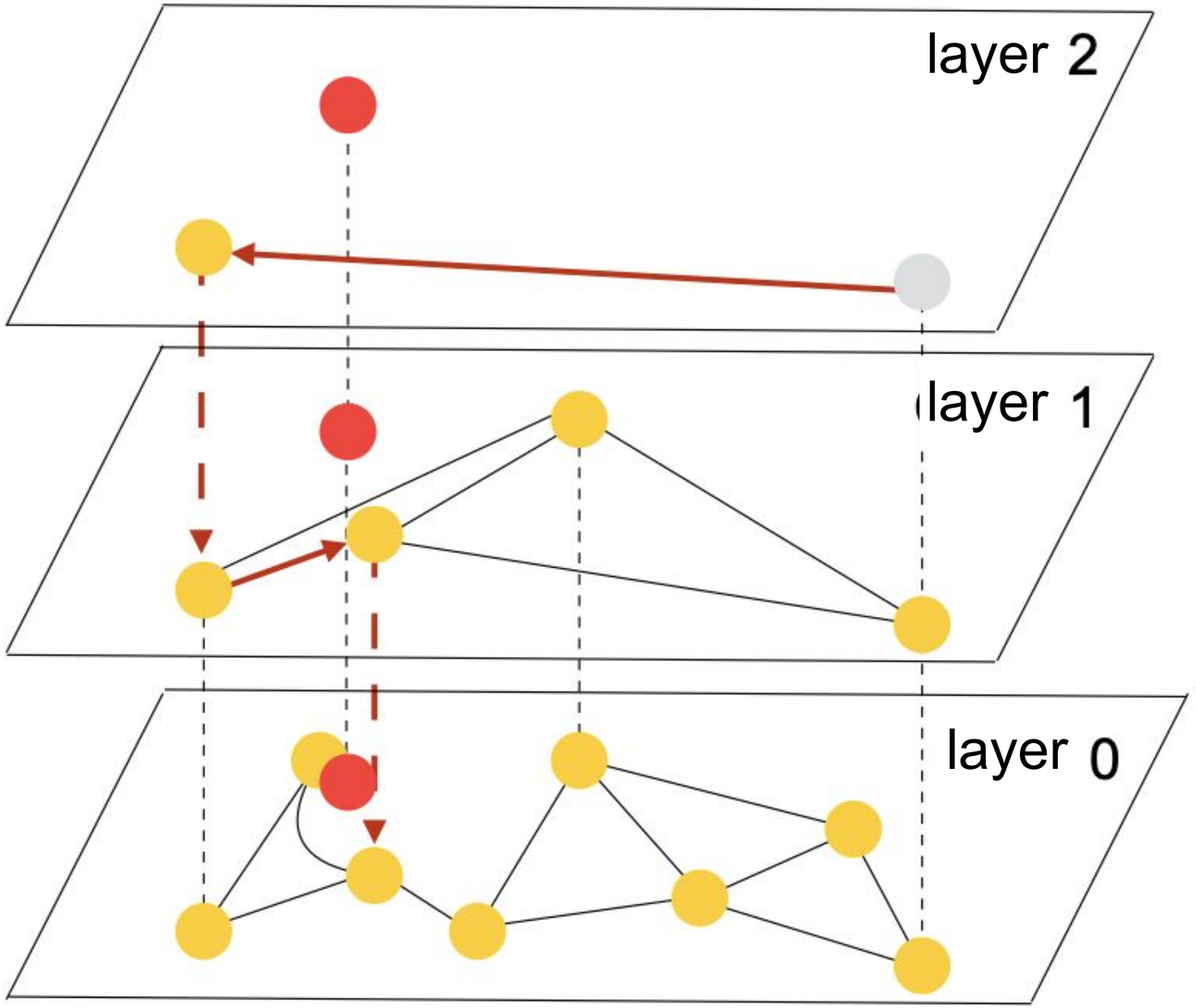
HNSW



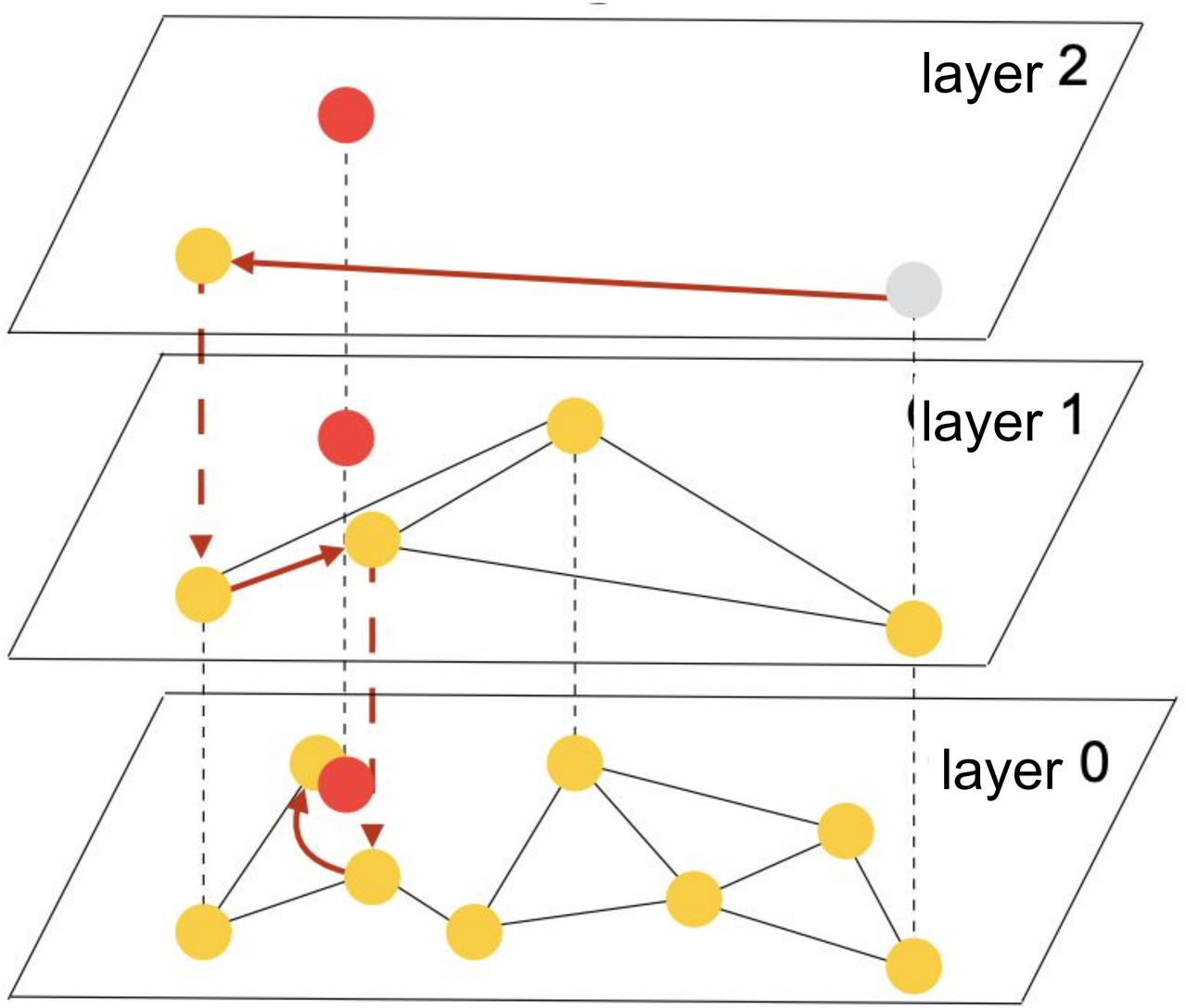
HNSW



HNSW

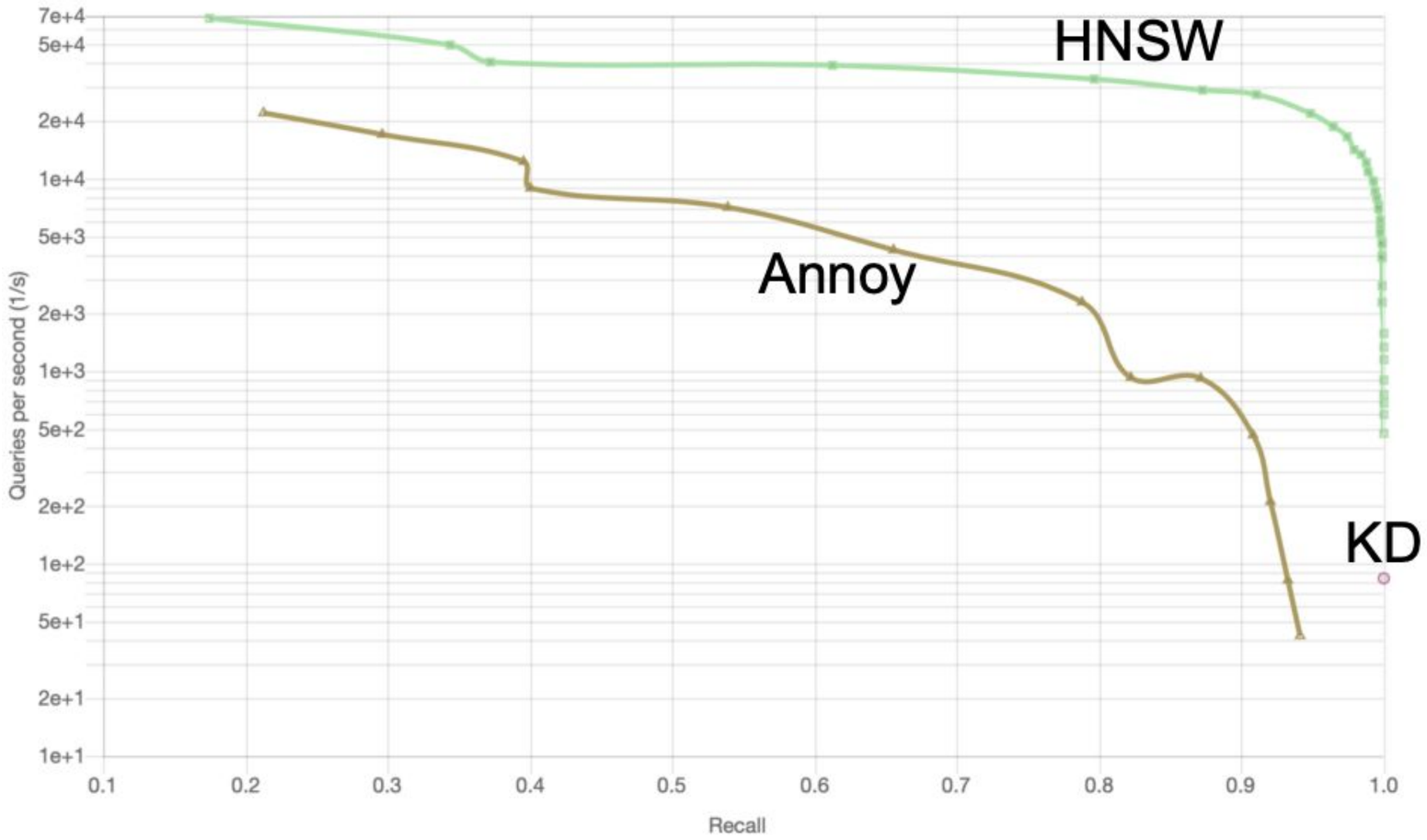


HNSW



Accuracy of approximate search

- In addition to speed, you need to find accurate nearby objects
- Comparison on ALS embedding last fm-64-dot, 360K
- https://github.com/erikbern/ann-benchmarks

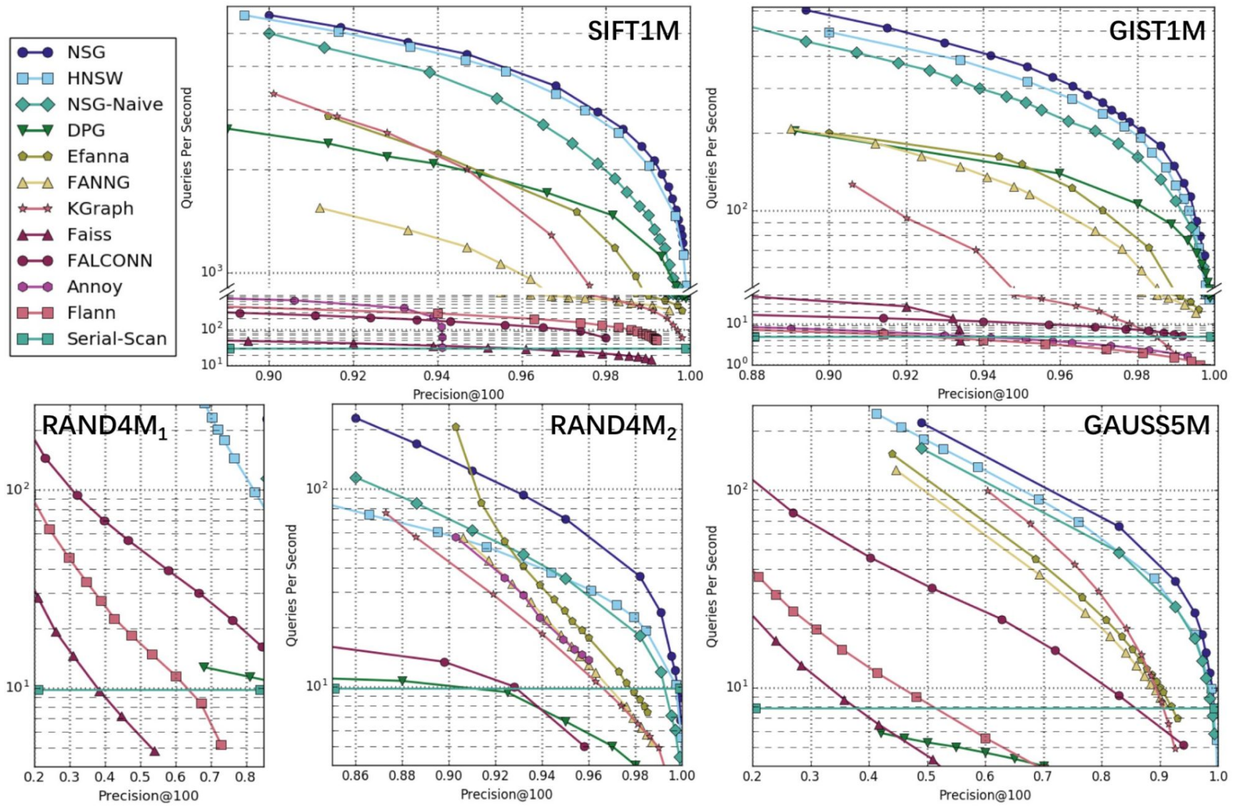


Navigating Spreading-out Graph

Another graph based approach which is even faster by recent benchmarks

https://arxiv.org/pdf/1707.00143

https://faiss.ai/cpp_api/struct/structfaiss_1_1Inde



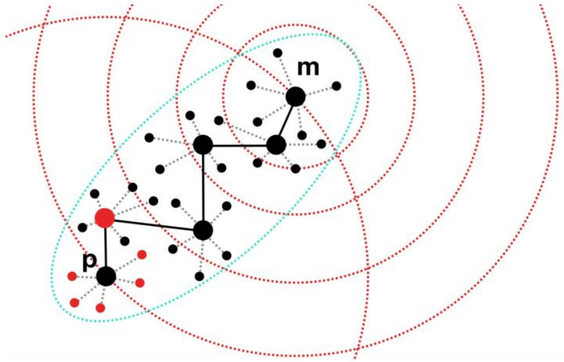


Figure 5: An illustration of the candidates of edge selection in NSG. Node p is the node to be processed, and m is the Navigating Node. The red nodes are the k nearest neighbors of node p. The big black nodes and the solid lines form a possible monotonic path from m to p, generated by the search-and-collect routine. The small black nodes are the nodes visited by the search-and-collect routine. All the nodes in the figure will be added to the candidate set of p.

ANN benchmarking arc

Period	Dominant question	Typical metrics	Representative benchmarks
1970s–1990s	Can exact indexing beat linear scan?	Query time, distance computations, node visits	kd-tree, M-tree, high-dimensional similarity studies
2000s	How much approximation is acceptable?	Recall, speedup, memory, parameter sensitivity	Liu et al. 2004, FLANN
2010s	Can we compare libraries fairly?	Recall–QPS curves, build time, index size	NMSLIB, Li et al. 2016, ANN-Benchmarks
Late 2010s	Can we search billion-scale datasets?	Recall@k, QPS, RAM, GPU/SSD use	PQ, Faiss GPU, DiskANN
2021–2023	Can we run fair public competitions under constraints?	Recall, latency, QPS, hardware/cost tracks	BigANN, SISAP challenges
2023–today	Can vector retrieval systems handle real application workloads?	Latency distributions, throughput, filtering, updates, cost, OOD behavior	VIBE, VectorDBBench, VSB, BigVectorBench

Selected benchmarks



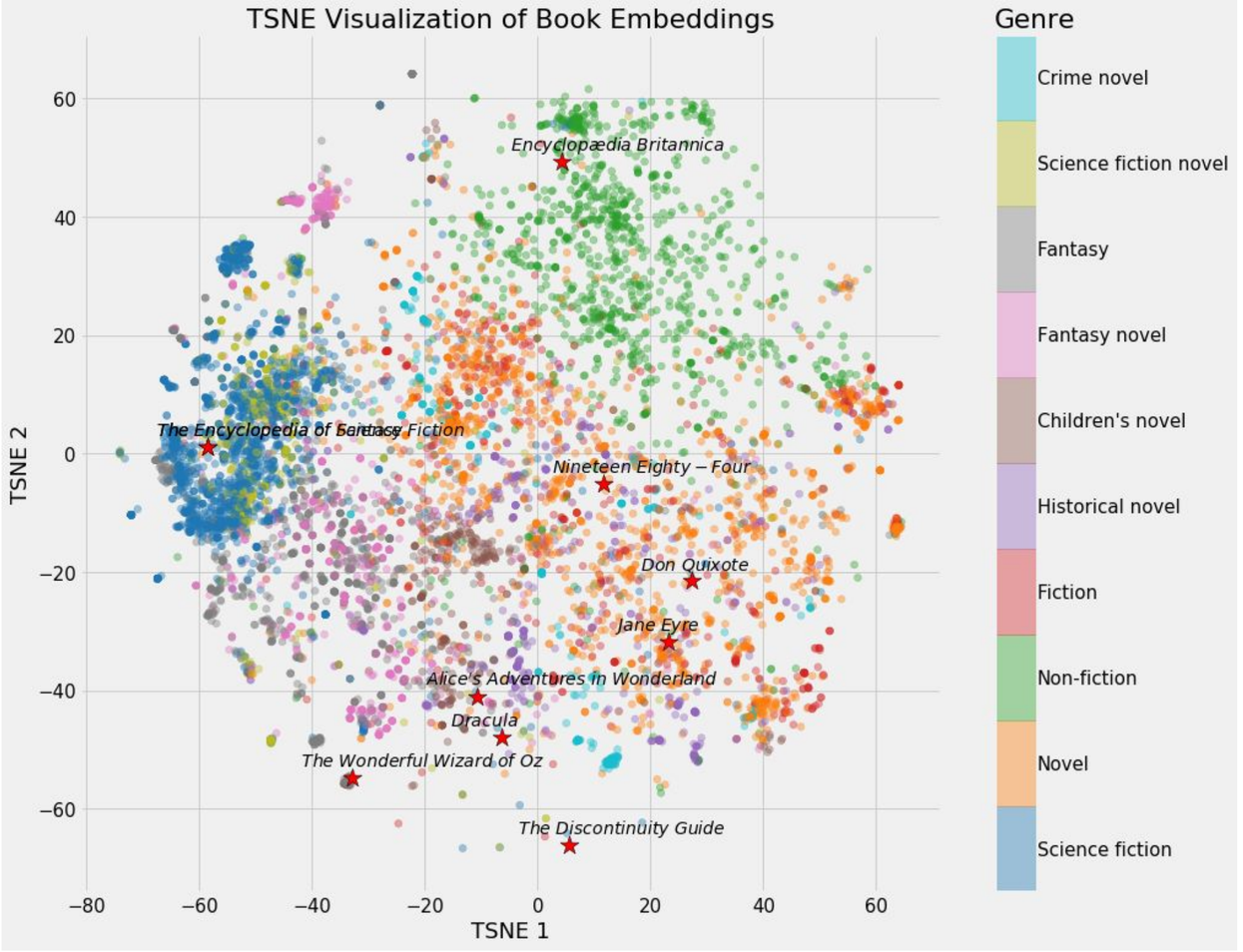
- Classical single machine benchmark
  - https://ann-benchmarks.com/index.html
  - https://github.com/erikbern/ann-benchmarks
- Qdrant benchmark - most advanced open source solution
  - https://qdrant.tech/benchmarks/
  - https://github.com/qdrant/vector-db-benchmark
- VectorDBBench by Zilliz (proprietary vector DB company)
  - https://zilliz.com/vdbbench-leaderboard?dataset=vectorSearch
  - https://github.com/zilliztech/vectordbbench
- Big-ANN benchmark
  - https://big-ann-benchmarks.com/neurips23.html
- Vector Index Benchmark for Embeddings (VIBE) - new fresh embedding first
  - https://vector-index-bench.github.io/overview.html
  - https://github.com/vector-index-bench/vibe

How ANN is used?

Main object to apply kNN nowadays are embeddings (vectors) produced by neural networks.

Such embeddings represent object properties in a compact format.

Nearest neighbors are used to roughly select possible candidates for further processing.



ANN usage examples



- Search Engines, Recommender systems
  - texts, images, videos, etc
- Bioidentification, Reidentification
  - by photo, 3d face scan, voice, iris, fingerprint, etc
- Activity Recognition, Anomaly Detection
  - transactions, network traffic, action on video, time series
- Camera Pose Estimation
- Protein Structure Prediction, Sequence Alignment
- Question Answering
- Retrieval Augmented Generation
- many more...

ANN implementations



- https://github.com/facebookresearch/faiss
  - https://github.com/facebookresearch/faiss/wiki
  - https://habr.com/ru/companies/okkamgroup/articles/509204/
- https://github.com/nmslib/nmslib
- https://github.com/flann-lib/flann
- https://github.com/spotify/annoy
- https://lucene.apache.org/core/9_1_0/core/org/apache/lucene/util/hnsw/package-summary.html

Vector databases



Also list of vector databases from openAI
https://platform.openai.com/docs/guides/embeddings/how-can-i-retrieve-k-nearest-embedding-vectors-quickly

Ready to production solutions

- FOSS
  - https://github.com/qdrant/qdrant
    - Has helm chart + distributed deployment
    - Fastest FOSS solution
  - https://github.com/feast-dev/feast
  - https://github.com/NVIDIA/aistore
- Proprietary
  - https://www.pinecone.io/
  - https://www.trychroma.com/
  - https://milvus.io/ by Zilliz

Sources



- https://vinija.ai/concepts/ann-similarity-search/
-

Facebook AI Similarity Search (FAISS)

Most used nowadays.

Contains all mainstream implementations of algorithms:

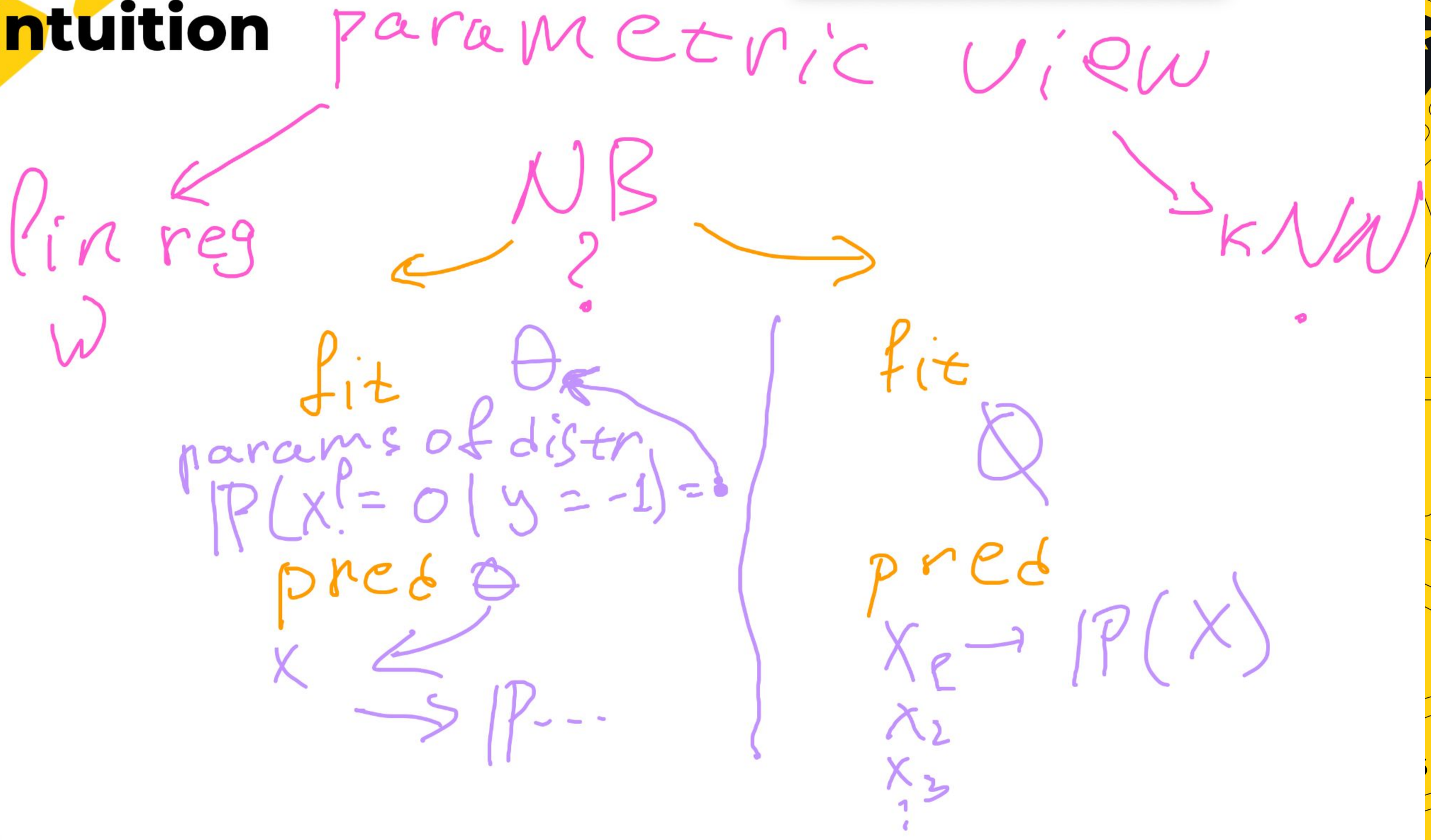
- bruteforce
- Inverted file search (IVF)
- Locality-Sensitive Hashing (LSH)
- Scalar quantizer (SQ)
- HNSW

https://github.com/facebookresearch/faiss/wiki/Faiss-indexes

ANN on GPU

- https://developer.nvidia.com/blog/accelerating-vector-search-fine-tuning-gpu-index-algorithms/
-

Intuition


parametric view
lin reg
w
NB
?
knn
fit
θ
params of distr
P(x^l = 0 | y = -1) =
pred
x
P...
fit
θ
pred
x_e
P(x)
x_2
x_3

tuition

lin reg

↓SVM

fit:

$L = w^T w + L_{hinge}$

pred

kNN

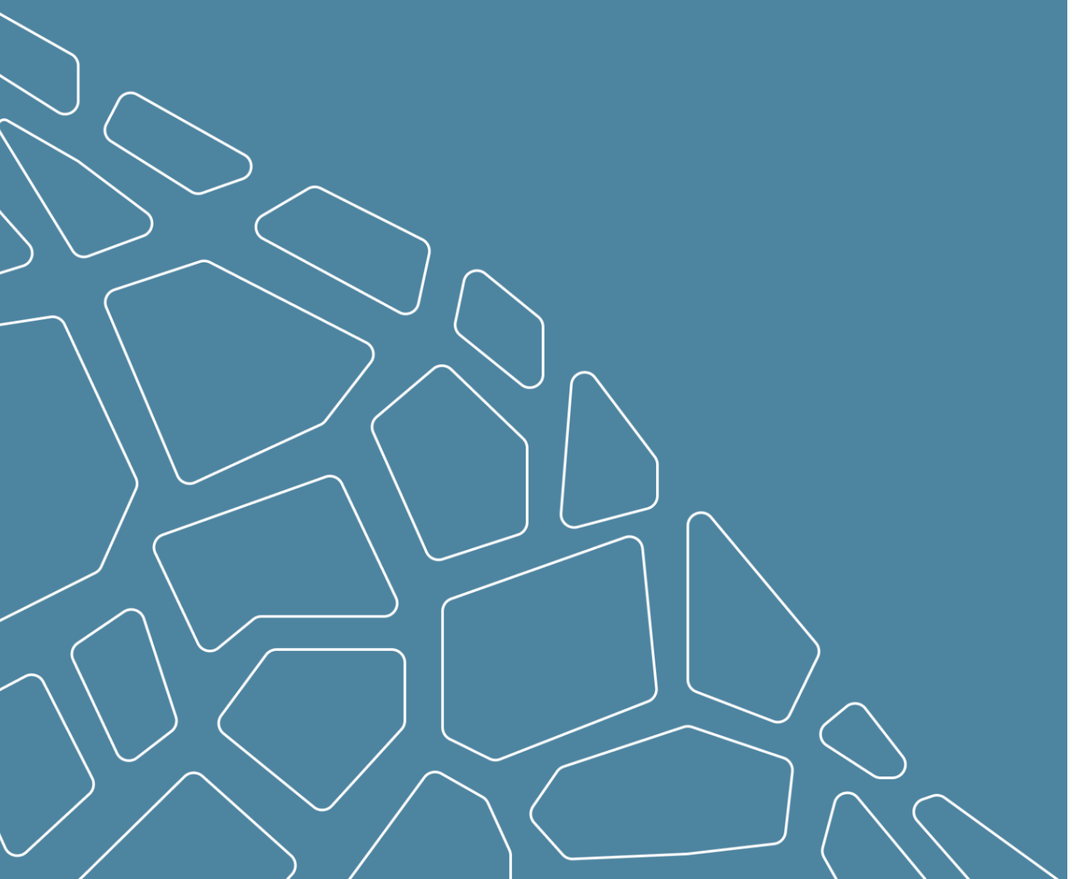
↓

fit: precomp HNSW graphs $\equiv \emptyset$

pred:

$\theta \rightarrow$ ans

Revise



1. k Nearest Neighbors model
    a. contemporary use cases
2. kNN indexes
    a. ideas
    b. Annoy
    c. HNSW
3. Vector databases
    a. Qdrant

Thanks for attention!

Questions?



girafe ai